

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

WEB TECHNOLOGIES - BITE304L

PROJECT REVIEW-01

FACULTY NAME: DR.VIJAYARANI A

PROJECT TITLE: ONLINE GROCERY SHOPPING MANAGEMENT

TEAM NO: 05

TEAM MEMBERS:

1. SUJAA SHRI S-23BIT0315
2. RAMYA M – 23BIT0329
3. THANUSHA E – 23BIT0346
4. ELAKIYA R – 23BIT0357
5. MUGESHWARAN E – 23BIT0307

ONLINE GROCERY SHOPPING MANAGEMENT

ABSTRACT:

Online grocery shopping has become an essential service in today's fast-paced world, offering convenience, efficiency, and accessibility. Consumers prefer the ease of ordering groceries online, saving time and avoiding long queues while accessing a wide range of products 24/7. With competitive pricing, personalized recommendations, and seamless payment options such as Net banking, credit/debit cards, UPI, and Cash on Delivery, online grocery platforms enhance cost-effective and hassle-free shopping. This project aims to develop an e-commerce website that provides a user-friendly shopping experience with a secure checkout process and efficient order management for administrators. The implementation involves multi-tiered architecture, server and client-side scripting, and technologies such as HTML, CSS, JavaScript. This document explores the development process, highlighting the key technologies and features that make online grocery shopping a necessity rather than a luxury.

MODULES:

Module 1: Customer Login

- The Customer selects **"Log In"** and enters their registered email/phone number and password.
- The system verifies the credentials and starts a session for the Customer.
- If the credentials are incorrect, an error message is displayed, allowing retry or password recovery.
- Once logged in, the Customer is redirected to the **homepage**, where they can browse products.
- If the Customer don't have an exsisting account,the Customer can create a new account

Module 2:Customer registration

- **Open Sign-Up Page** – Customer selects **"Sign Up"** on the website or app.
- **Enter Details** – Customer provides:**Full Name ,Email Address,Password** (must meet security requirements)
- **Validate Input** – System checks for:

- Missing or invalid email format.
- Weak password (e.g., minimum characters, special symbols).
- **Verify Email** – System sends an **OTP (One-Time Password)** to the email for verification.
- **Enter OTP** – Customer inputs the received OTP to confirm email ownership.
- **Account Creation** – System stores Customer details securely and creates the account.
- **Confirmation & Login** – Customer sees a success message and is redirected to the **login page**.

Module 3: Product Catalog

- The Customer navigates through different grocery categories such as **Fruits & Vegetables, home&kitchen,packed food,personal care.**
- They can use the **search bar** to find specific products and apply **filters** (e.g., price range).
- Clicking on a product displays **detailed information** like price, weight, description, and stock availability.

Module 4: Adding Products to the Shopping Cart

- The Customer selects a product and clicks **"Add to Cart."**
- The system updates the **shopping cart**, displaying the selected items and their quantities.
- The Customer can:
    - Increase or decrease the product quantity.
    - Can delete item
    - Continue shopping or proceed to checkout.

Module 4: Proceeding to Checkout

- When ready, the Customer clicks **"Checkout."**
- The system asks the Customer to confirm:
    - Delivery address (can be changed or added at this stage).
    - Payment method (choose from saved options or enter new details).
- The system calculates the **total amount, including taxes and delivery charges.**
- A final order summary is displayed before payment.

Module 5: Order View

- Customer logs in and navigates to **"My Orders."**
- The system displays a **list of past and current orders**, showing:
    - **Order ID**
    - **Order Date & Time**
    - **Items Purchased**

    - **Total Amount Paid**
    - **Payment Method** (Card, UPI, Cash on Delivery)
    - **Order Status** (Processing, Shipped, Delivered, Canceled)
- Customer can click on an order to view detailed information, including invoices and delivery addresses.

Module 6: Order Tracking

- Customer selects an active order to track its progress.
- The system provides **real-time tracking updates**, such as:
    - **Order Placed** – Confirmation received.
    - **Processing** – Items being packed.
    - **Shipped** – Out for delivery with estimated arrival time.
    - **Delivered** – Successfully delivered.

Module 7: Order Cancellation

- Customer navigates to **"My Orders"** and selects an order that is still in **Processing** or **Not Yet Shipped** status.
- Clicks on **"Cancel Order."**
- System prompts for **cancellation reason** .
- If eligible for cancellation:
    - Order is canceled, and the Customer receives a **confirmation message**.
    - Refund is initiated (if paid online), and the amount is processed based on the payment method.
    - If it was **Cash on Delivery (COD),** no refund is required.
- If the order is **already shipped or out for delivery,** cancellation is not allowed.
- The system updates the order status to **"Canceled"** in the order history.

Module 8: Payment Processing

- The Customer selects a **payment method**:
    - Credit/Debit Card
    - UPI
    - Net Banking
    - Cash on Delivery
- If using an online payment method, the system redirects to the payment gateway for processing.
- Upon successful payment, the system:

    - Confirms the order.
    - Generates an **order receipt** and sends it via email/SMS.
    - Updates the order status to **"Processing."**
- If payment fails, the Customer is notified and given options to retry or change the payment method.

Module:9 Profile management

- The Customer logs in and navigates to **"My Profile"** .
- The system displays Customer details, including name, email, phone number, saved addresses, payment methods, and order history.
- The Customer selects **"Edit Profile"** and updates their name, email (may require OTP verification), or phone number.
- After clicking **"Save Changes,"** the system validates the input and updates the details.
- The Customer selects **"Manage Addresses"** to add, edit, or delete delivery addresses.
- The system verifies the address format before saving changes.
- The Customer selects **"Change Password"** and enters their current password, followed by a new one.
- The system validates the new password and updates it securely.

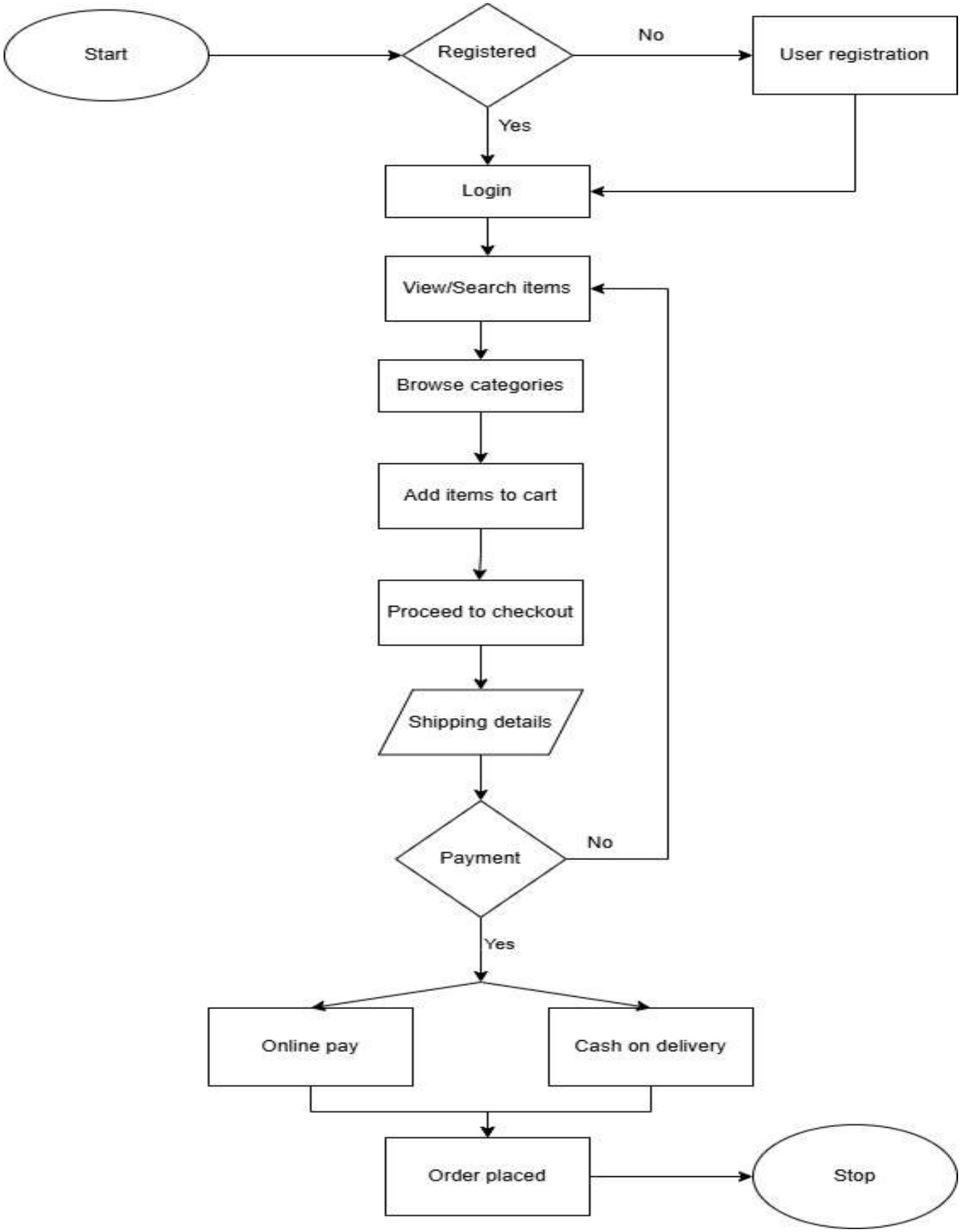
Module 10: Logging Out

- Once done, the Customer can click **"Logout."**
- The system:
    - Ends the session.
    - Clears temporary data.
    - Redirects the Customer to the login/homepage.

Module 11: Footer Module

- About Us: Brief store overview, mission, quality assurance, and company values.
- Contact Us: Inquiry form, contact details, social media links, and FAQs.
- Cancellation Policy: Order cancellation rules, refund process, and eligibility conditions.

PROCESS FLOW DIAGRAM:



DATABASE TABLES:

1. Users Table

Column Name	Data Type	Constraints	Description
customer_id	INT	PRIMARY KEY,AUTO_INCREMENT	Unique User ID of user
name	VARCHAR(100)	NOT NULL	Full name of the user
email	VARCHAR(100)	UNIQUE, NOT NULL	Email ID
password	VARCHAR(255)	NOT NULL	Encrypted user password
phone	CHAR(10)	UNIQUE	Contact number
address	TEXT		User's full address

2. Products Table:

Column Name	Data Type	Constraints	Description
product_id	INT	PRIMARY KEY	Unique identifier for product
name	VARCHAR(100)	NOT NULL	Product name
description	TEXT		Detailed product description
price	DECIMAL(10,2)	NOT NULL	Product price
category_id	INT	FOREIGN KEY REFERENCES categories(category_id)	Product category

Current_price	DECIMAL(10,2)	NOT NULL	Product price
subcat	VARCHAR(100)	CHECK(subcat in (SELECT subcategories FROM Categories))	Product Subcategory
stock_quantity	INT	NOT NULL	Available stock
image_url	VARCHAR(255)	NOT NULL	Image path for product

3. Categories table:

Column Name	Data Type	Constraints	Description
category_id	INT	PRIMARY KEY	Unique category ID
category_name	VARCHAR(100)	UNIQUE, NOT NULL	Category name (e.g., Dairy, Vegetables)
subcategories	ENUM	NOT NULL	Subcategories of each category

4. Orders Table

Column Name	Data Type	Constraints	Description
order_id	INT	PRIMARY KEY	Unique order ID
customer_id	INT	FOREIGN KEY REFERENCES users(customer_id)	Customer placing the order
name	VARCHAR(100)	NOT NULL	Full name of the user
phone	CHAR(10)	UNIQUE,NOT NULL	Contact number
address	TEXT	NOT NULL	User's full address

total	DECIMAL(10,2)	NOT NULL	Total order amount
payment_status	VARCHAR(10)	FOREIGN KEY (payments(payment_status))	Payment status
status	ENUM('placed', 'shipped', 'delivered', 'cancelled')	DEFAULT 'placed'	Status of the order
Placed_at	TIMESTAMP	DEFAULT TIMESTAMP	Date and time when order placed
items	JSON objects		Ordered items

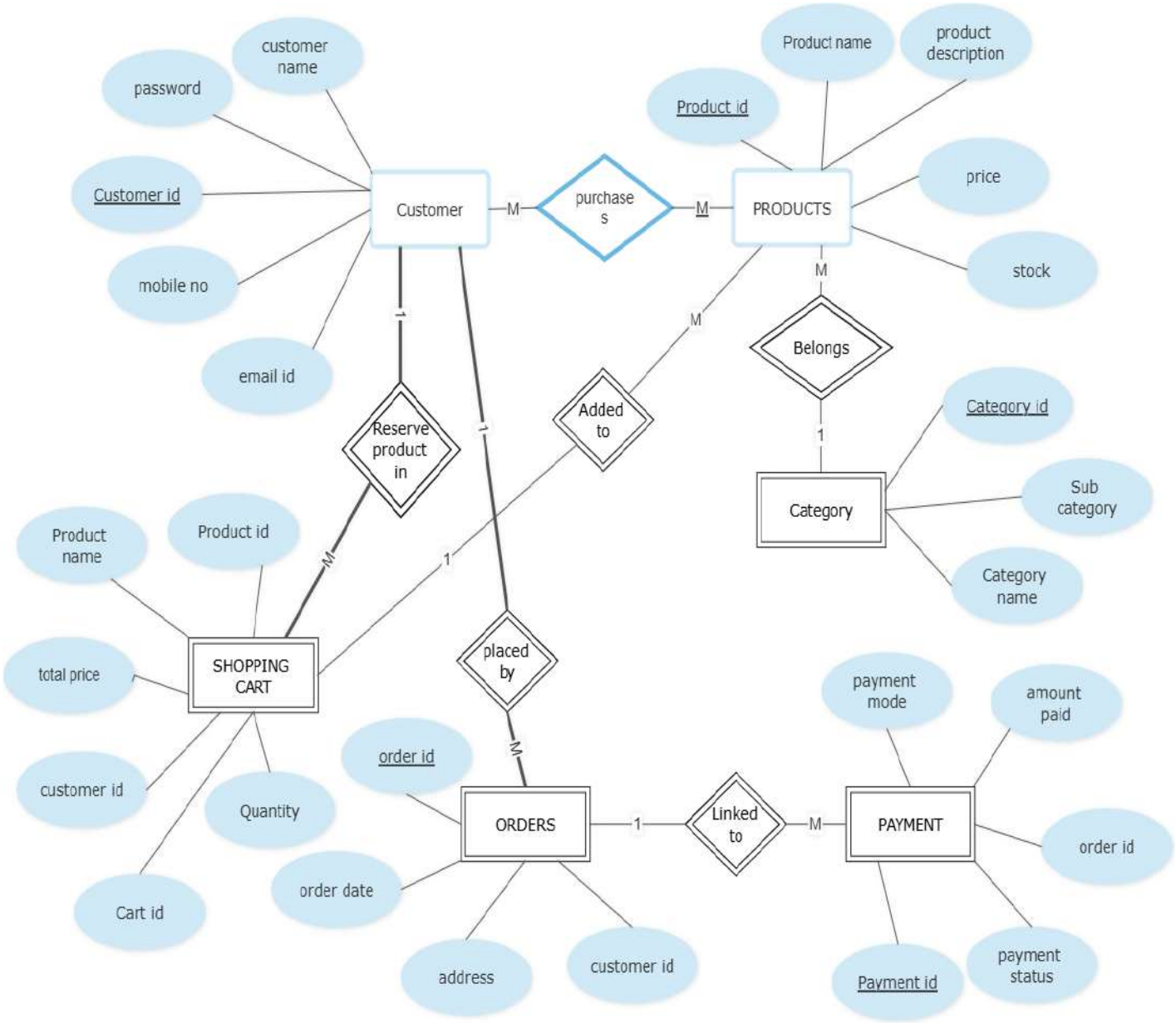
5. Payments table:

Column Name	Data Type	Constraints	Description
payment_id	INT	PRIMARY KEY	Unique payment ID
order_id	INT	FOREIGN KEY REFERENCES orders(order_id)	Associated order
payment_method	ENUM('credit/debitcard', 'net_banking', 'UPI', 'COD')	NOT NULL	Payment type used
amount_paid	DECIMAL(10,2)	NOT NULL	Amount paid for the order
payment_status	VARCHAR(10)	CHECK(payment_status in ('completed', 'failed')), DEFAULT 'failed'	Payment status
payment_time	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Date of payment

6. Cart table

Column Name	Data Type	Constraints	Description
cart_id	INT	PRIMARY KEY	Unique cart ID
customer_id	INT	FOREIGN KEY REFERENCES users(customer_id)	User who manages the cart
product_id	INT	FOREIGN KEY REFERENCES products(product_id)	Product in cart
price	DECIMAL(10,2)	NOT NULL	Product price
quantity	INT	NOT NULL	Quantity of product
product_name	VARCHAR(255)	NOT NULL	Name of the products

ER DIAGRAM:


customer name
password
Customer id
mobile no
email id
Customer
M
purchase s
M
PRODUCTS
Product name
product description
Product id
price
stock
M
Belongs
1
Category id
Sub category
Category name
Category
M
Added to
1
Reserve product in
1
1
M
Product name
Product id
total price
SHOPPING CART
customer id
Quantity
Cart id
placed by
M
order id
ORDERS
order date
address
customer id
1
Linked to
M
payment mode
amount paid
PAYMENT
order id
Payment id
payment status

CODE:

USERS PAGE :

HOMEPAGE : homepg.html

```
<html>
<head>
  <title>QuickMart - Grocery Store</title>
  <link rel="stylesheet" href="cat.css">
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
  <script src="temp.js"></script>

</head>
<body>

  <nav class="topnav">
    <div class="logo">QuickMart</div>
    <input type="text" id="loc" placeholder="Enter Pincode" maxlength="6">
    <a class="NavLogo" href="homepg.html">Home 🏠 </a>
    <input type="text" id="search-bar" placeholder="Search QuickMart"
onkeyup="redirectToSearch(event)">
    <div class="user">
      <a class="NavLogo" href="login2.html">Login 👤 </a>
    </div>
    <div class="cart">
      <a class="NavLogo" href="cart2.html">Cart 🛒 <span id="cart-count"> </span></a>
    </div>
    <div class="nav3" id="menu-toggle">☰</div>
  </nav>
  <div id="sidebar">
```

```
    <a href="profile.html" class="sidebar-link">My Profile</a>
    <a href="myorders.html" class="sidebar-link">My Orders</a>
    <a href="#" class="sidebar-link" id="logout">Logout</a>
  </div>
  <div class="categories-nav">
    <a href="fruits.html">Fruits & Vegetables</a>
    <a href="home.html">Home & Kitchen</a>
    <a href="packaged.html">Packaged Foods</a>
    <a href="personalcare.html">Personal Care</a>
  </div>

  <div class="banner"><img src="ig.jpg" ></div>

  <div class="category-container">
    <a href="fruits.html" class="category2">
      <img src="temp8.jpg" alt="Fruits">
    </a>
    <a href="home.html" class="category2">
      <img src="temp2.jpg" alt="Vegetables">
    </a>
    <a href="packaged.html" class="category2">
      <img src="temp7.jpg" alt="Dairy Products">
    </a>
    <a href="PersonalCare.html" class="category2">
      <img src="temp6.jpg" alt="Snacks">
    </a>
  </div>

  <div class="footer">
```

```
    <a href="aboutus.html">About Us</a>
    <a href="contactus.html">Contact Us</a>
    <a href="cancellation.html">Cancellation Policy</a>
  </div>

</body>
</html>
```

SEARCHBAR : searchbar.html

```
<html>
  <head>
    <title>Search</title>
    <link rel="stylesheet" href="cat.css">
    <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
    <script src="temp.js"></script>
    <script>
      $(document).ready(() => {
  function fetchSearchResults() {
    let searchParams = new URLSearchParams(window.location.search);
    let query = searchParams.get("q");

    if (!query) {
      $(".products").html("<p>No search query provided</p>");
      return;
    }

    $(".products").html("<p>Searching...</p>");

    $.get(`/products?search=${encodeURIComponent(query)}`, (products) => {
      $(".products").empty();
```

```
        if (products.length === 0) {
          $(".products").html(` <p>No results found for '${query}'</p>`);
          return;
        }

        products.forEach((prod) => {
          let discount = Math.round(((prod.price - prod.current_price) / prod.price) * 100);
          let productHTML = `
          <div class="product-box" data-product-id="${prod._id}">
            <img src="${prod.image_url}" class="prod">
            <p>${prod.name}<br>
              <s>₹${prod.price}</s>  
              <span class="disc">${discount}% OFF</span><br>
              ₹<span class="current-price">${prod.current_price}</span>
            </p>
            <button class="atc">Add to cart</button>
          </div>`;

          $(".products").append(productHTML);
        });
      }).fail((err) => {
        console.error("Error fetching search results:", err);
        $(".products").html("<p>Error fetching search results</p>");
      });
    }

    fetchSearchResults();
});
```

```
    </script>
  </head>
  <body>
    <div><nav class="topnav">
      <div class="logo">QuickMart</div>
      <input type="text" id="loc" placeholder="Enter Pincode" maxlength="6">
      <a class="NavLogo" href="homepg.html">🏠 Home</span></a>
      <input type="text" id="search-bar" placeholder="Search QuickMart"
onkeyup="redirectToSearch(event)">
      <div class="user">
        <a class="NavLogo" href="login2.html">Login 👤 </a>
      </div>
      <div class="cart">
        <a class="NavLogo" href="cart2.html">Cart 🛒 <i class="shopping-cart"></i> <span id="cart-
count"></span></a></button>
      </div>
      <div class="nav3" id="menu-toggle">☰</div>
    </nav>
    <div id="sidebar">
      <a href="profile.html" class="sidebar-link">My Profile</a>
      <a href="myorders.html" class="sidebar-link">My Orders</a>
      <a href="#" class="sidebar-link" id="logout">Logout</a>
    </div>
    </div>
    <div class="categories-nav">
      <a href="fruits.html">Fruits & Vegetables</a>
      <a href="home.html">Home & Kitchen</a>
      <a href="packaged.html">Packaged Foods</a>
      <a href="personalcare.html">Personal Care</a>
    </div>
```

```
    <div class="products">


    </div>
  </div>
</body>
</html>
```

SIGN UP AND LOGIN PAGE:

signup.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Signup</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      display: flex;
      flex-direction: column;
      align-items: center;
      justify-content: center;
      height: 100vh;
      background-color: #f8f8f8;
    }
    .signup-container {
      text-align: center;
    }
```

```
.signup-box {
   width: 400px;
   height: 550px;
   background: white;
   padding: 40px;
   text-align: center;
   border-radius: 5px;
   border: 1px solid black;
}
h2 {
font-size: 22px;
margin-bottom: 15px;
}
input {
   width: 80%;
   padding: 10px;
   margin-top: 5px;
   border: 1px solid #aaa;
   margin-bottom: 10px;
   font-size: 15px;
}
button {
   width: 70%;
   padding: 10px;
   background: rgb(6,147,65);
   color: white;
   font-weight: bold;
   border-radius: 10px;
   border: none;
```

```
    cursor: pointer;
    text-align: center;
    font-size: 18px;
    margin-top: 10px;
}
.error {
    color: red;
    font-size: 14px;
}
button:hover {
background: rgb(159, 217, 183);
}
button:active{
transform: scale(0.95);
}
.input-container {
position: relative;
width: 100%;
}
.password-info1{
font-size: 15px;
color: gray;
margin-top: 0px;
margin-right: 60px;
}
.password-info2{
font-size: 15px;
color: gray;
margin-top: 0px;
```

```
text-align: left;
margin-left: 35px;
}
.toggle-icon1{
position: absolute;
right: 42px;
top: 20%;
transform: translateY(-50%);
width: 35px;
cursor: pointer;
}
.toggle-icon2{
position: absolute;
right: 42px;
top: 40%;
transform: translateY(-50%);
width: 35px;
cursor: pointer;
}
.signin-text{
 margin-top: 20px;
}
.login-text{
color: #0066c0;
text-decoration: none;
}
.login-text:hover{
text-decoration: underline;
}
```

```
  </style>
</head>
<body>
  <div class="signup-container">
    <h1 style="color: rgb(6,147,65)">QuickMart</h1>
    <div class="signup-box">
      <h2>Create Account</h2>
      <form id="signup-form">
        <input type="text" id="name" placeholder="Full Name" required>
        <input type="email" id="email" placeholder="Email" required>
        <div class="input-container">
        <input type="password" id="password" placeholder="Password" required>
        <p class="password-info1">Passwords must be at least 8 characters.</p>
        <p class="password-info2">Password must include at least one uppercase letter, one lowercase letter, one number, and one special character.</p>
        <img src="pass_show.jpg" alt="Show Password" id="toggleIcon1" class="toggle-icon1" onclick="togglePassword1()">
        </div>

        <div class="input-container">
        <input type="password" id="confirm-password"placeholder="Confirm Password" required>
        <img src="pass_show.jpg" alt="Show Password" id="toggleIcon2" class="toggle-icon2" onclick="togglePassword2()">
        </div>
        <p id="error-message" class="error"></p>
        <button type="submit">SIGN UP</button>
      </form>
    </div>
    <p class="signin-text">Already have an account? <a class="login-text" href="login2.html">Log In</a></p>
  </div>
```

```
<script>
document.addEventListener("DOMContentLoaded", function () {
  const signupForm = document.getElementById("signup-form");
  const errorMessage = document.getElementById("error-message");

  signupForm.addEventListener("submit", async function (event) {
    event.preventDefault();
    errorMessage.textContent = "";

    let fullName = document.getElementById("name").value.trim();
    let email = document.getElementById("email").value.trim();
    let password = document.getElementById("password").value.trim();
    let confirmPassword = document.getElementById("confirm-password").value.trim();

    let emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    let passwordRegex = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[!@#$%^&*()_+\-=\[\]{}|]).{8,}$/;

    if (!fullName || !email || !password || !confirmPassword) {
      errorMessage.textContent = "All fields are required";
      return;
    }
    if (!emailRegex.test(email)) {
      errorMessage.textContent = "Enter a valid email";
      return;
    }
    if (password.length < 8) {
      errorMessage.textContent = "Password must be at least 8 characters long";
      return;
```

```
        }

        if (password.length > 128) {
            errorMessage.textContent = "Password must not exceed 128 characters";
            return;
        }
        if (!passwordRegex.test(password)) {
            errorMessage.textContent = "Password must include at least one uppercase, one lowercase, one number and one special character";
            return;
        }
        if (password !== confirmPassword) {
            errorMessage.textContent = "Passwords do not match";
            return;
        }

        try {
            let response = await fetch("http://localhost:5000/signup", {
                method: "POST",
                headers: { "Content-Type": "application/json" },
                body: JSON.stringify({ name: fullName, email, password })
            });

            let result = await response.json();
            if (response.ok) {
                alert("Signup Successful");
                window.location.href = "login2.html";
            } else {
                errorMessage.textContent = result.error;
```

```
        }
    } catch (error) {
        errorMessage.textContent = "Error signing up.try again.";
    }
  });
});
  function togglePassword1() {
    let passwordInput = document.getElementById("password");
    let icon = document.getElementById("toggleIcon1");

    if (passwordInput.type === "password") {
        passwordInput.type = "text";
        icon.src = "pass_hide.jpg";
        icon.alt = "Hide Password";
    } else {
        passwordInput.type = "password";
        icon.src = "pass_show.jpg";
        icon.alt = "Show Password";
    }
  }
  function togglePassword2() {
    let passwordInput = document.getElementById("confirm-password");
    let icon = document.getElementById("toggleIcon2");

    if (passwordInput.type === "password") {
        passwordInput.type = "text";
        icon.src = "pass_hide.jpg";
        icon.alt = "Hide Password";
    } else {
```

```
            passwordInput.type = "password";
            icon.src = "pass_show.jpg";
            icon.alt = "Show Password";
        }
    }
  </script>
</body>
</html>
```

Login2.html:

```
<html>
<head>
  <title>Login</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      display: flex;
      flex-direction: column;
      align-items: center;
      justify-content: center;
      height: 100vh;
      background-color: #f8f8f8;
    }
    .login-container {
      text-align: center;
    }
    .login-box {
      width: 350px;
      height: 380px;
      background: white;
```

```
    padding: 40px;
    text-align: center;
    border-radius: 5px;
    border: 1px solid black;
    position: relative;
}
h2 {
    font-size: 25px;
    margin-bottom: 15px;
    text-align: center;
}
.input-container {
    position: relative;
    width: 100%;
}
input {
    width: 90%;
    padding: 10px;
    margin-top: 5px;
    border: 1px solid #aaa;
    border-radius: 0px;
    margin-bottom: 15px;
    font-size: 15px;
}
.toggle-icon {
    position: absolute;
    right: 20px;
    top: 40%;
    transform: translateY(-50%);
```

```
    width: 35px;
    cursor: pointer;
}
.forpass {
    display: block;
    font-size: 14px;
    margin-top: 10px;
    margin-left: 10px;
    color: #0066c0;
    text-decoration: none;
    text-align: left;
}
.forpass:hover {
    text-decoration: underline;
}
button {
    width: 60%;
    padding: 10px;
    background: rgb(6,147,65);
    color: white;
    border: none;
    border-radius: 10px;
    cursor: pointer;
    justify-content: center;
    align-items: center;
    font-weight: bold;
    font-size: 18px;
}
button:active {
```

```
        transform: scale(0.95);
    }
    button:hover {
        background-color: rgb(159, 217, 183);
    }
    .new-account {
        margin-top: 20px;
    }
    .create-account {
        width: 100%;
        background: rgb(6,147,65);
        border: 1px solid #aaa;
        color: white;
    }
    .create-account:hover {
        background: rgb(159, 217, 183);
    }
    .start-shopping {
            display: inline-block;
            padding: 10px 20px;
            background-color: #007bff;
            color: white;
            text-decoration: none;
            border-radius: 5px;
            margin-top: 20px;
        }
  </style>
</head>
<body>
```

```
  <div class="login-container">
    <h1 style="color: rgb(6,147,65)">QuickMart</h1>
    <div class="login-box">
      <h2>LOGIN</h2>
      <form id="login-form">
        <input type="text" id="email" required placeholder="Email or Mobile Number">

        <div class="input-container">
          <input type="password" id="pass" required placeholder="Password">
          <img src="pass_show.jpg" alt="Show Password" id="toggleIcon" class="toggle-icon"
onclick="togglePassword()">
        </div>

        <a href="forgotpass.html" class="forpass">Forgot Password?</a><br>
        <button type="submit">LOGIN</button>
      </form>
    </div>
    <div class="new-account">
      <p>Don't have an account?</p>
      <button class="create-account" onclick="redirectToSignup()">SIGN UP</button>
    </div>
  </div>
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
  <script>
    $(document).ready(function() {
  $.get("http://localhost:5000/session", function(data) {
    if (data.loggedIn) {
      $("body").html(`
      <h2>User already Logged in</h2>
```

```
        <a href="homepg.html" class="start-shopping">Back to Home</a>
        `);
      } else {
        $("#login-form").show();
      }
    }).fail(function() {
      console.error("Error checking session");
    });
});

  document.addEventListener("DOMContentLoaded", function () {
    const loginForm = document.getElementById("login-form");

    loginForm.addEventListener("submit", async function (event) {
      event.preventDefault();

      let userInput = document.getElementById("email").value.trim();
      let password = document.getElementById("pass").value.trim();

      try {
        const response = await fetch("http://localhost:5000/login2", {
          method: "POST",
          headers: { "Content-Type": "application/json" },
           credentials: "include",
          body: JSON.stringify({ email: userInput, password: password })
        });

        const data = await response.json();
```

```
        if (response.ok) {
          alert("Login Successful");
          window.location.href = "homepg.html";
        } else {
          alert(data.message);
        }
      } catch (error) {
        alert("Server error.try again.");

      }
    });
});

function redirectToSignup() {
  window.location.href = "signupfinal.html";
}

function togglePassword() {
  let passwordInput = document.getElementById("pass");
  let icon = document.getElementById("toggleIcon");

  if (passwordInput.type === "password") {
    passwordInput.type = "text";
    icon.src = "pass_hide.jpg";
    icon.alt = "Hide Password";
  } else {
    passwordInput.type = "password";
    icon.src = "pass_show.jpg";
    icon.alt = "Show Password";
```

```
    }
  }


  </script>
</body>
</html>
```

Forgot password: forgot.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Forgot Password - QuickMart</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      display: flex;
      flex-direction: column;
      align-items: center;
      justify-content: center;
      height: 100vh;
      background-color: #f8f8f8;
      margin: 0;
    }


    .container {
      width: 400px;
      height:450px;
      background: white;
      padding: 30px;
```

```
    text-align: center;
    border-radius: 5px;
    border: 1px solid black;
    display: none;
}

h1 {
    color: rgb(6,147,65);
}

h2 {
    font-size: 25px;
    margin-bottom: 15px;
}

input {
    width: 90%;
    padding: 10px;
    margin: 10px 0;
    border: 1px solid #aaa;
    font-size: 15px;
}

button{
    width: 70%;
    padding: 10px;
    background: rgb(6,147,65);
    color: white;
    font-weight: bold;
```

```
        border: none;
        border-radius: 10px;
        cursor: pointer;
        font-size: 18px;
        margin-top: 100px;
    }



    button:hover {
        background-color: rgb(159, 217, 183);
    }

    button:active {
        transform: scale(0.95);
    }

    form .input-field {
  flex-direction: row;
  column-gap: 10px;
}
.input-field input {
  height: 40px;
  width: 40px;
  border-radius: 6px;
  outline: none;
  font-size: 1.125rem;
  text-align: center;
  border: 1px solid black;
}
```

```
 .input-field input:focus {
   box-shadow: 0 1px 0 rgba(0, 0, 0, 0.1);
 }
 .input-field input::-webkit-inner-spin-button,
 .input-field input::-webkit-outer-spin-button {
   display: none;
 }
 .error {
        color: red;
        font-size: 14px;
     }
 .toggle-icon1{
   position: absolute;
   right: 15px;
   top: 20%;
   transform: translateY(-50%);
   width: 35px;
   cursor: pointer;
}
.toggle-icon2{
   position: absolute;
   right: 15px;
   top: 50%;
   transform: translateY(-50%);
   width: 35px;
   cursor: pointer;
}

      .input-container {
```

```
        position: relative;
        width: 90%;
        margin: auto;
      }

      .rese {
        color: #0066c0;
        text-decoration: none;
      }

      .rese:hover {
        text-decoration: underline;
      }

      .password-info1{
   font-size: 15px;
   color: gray;
   margin-top: 0px;
   margin-right: 60px;
}
.password-info2{
   font-size: 15px;
   color: gray;
   margin-top: 0px;
   text-align: left;
   margin-left: 10px;
}

   </style>
```

```
</head>
<body>
  <h1>QuickMart</h1>

  <!-- Step 1: Forgot Password -->
  <div class="container" id="step1">
    <h2>Forgot Password</h2><br><br><br><br>
    <form id="forgot-form">
      <input type="text" id="email" placeholder="Email ID " required>
      <button type="submit">Continue</button>
    </form>
  </div>

  <!-- Step 2: OTP Verification -->
  <div class="container" id="step2">
    <h2>OTP</h2>
    <p>Enter the verification code sent to your Email ID</p>
    <form id="otp-form">
      <div class="input-field">
        <input type="text" maxlength="1" class="otp-input" required>
        <input type="text" maxlength="1" class="otp-input" disabled required>
        <input type="text" maxlength="1" class="otp-input" disabled required>
        <input type="text" maxlength="1" class="otp-input" disabled required>
      </div>
      <p>Didn't receive the code? <a href="#" class="rese" id="resend-btn">Resend</a></p>
      <button type="submit" id="verify-btn" disabled>Verify</button>
    </form>
  </div>
```

```
  <!-- Step 3: Reset Password -->
  <div class="container" id="step3">
    <h2>Reset Password</h2>
    <form id="reset-form">
      <div class="input-container">
        <input type="password" id="password" placeholder="Password" required>
        <p class="password-info1">Passwords must be at least 8 characters.</p>
          <p class="password-info2">Password must include at least one uppercase letter, one lowercase letter, one number, and one special character.</p>
          <img src="pass_show.jpg" alt="Show Password" id="toggleIcon1" class="toggle-icon1" onclick="togglePassword1()">
      </div>
      <div class="input-container">
        <input type="password" id="confirm-password" placeholder="Confirm Password" required>
        <img src="pass_show.jpg" alt="Show Password" id="toggleIcon2" class="toggle-icon2" onclick="togglePassword2()">
      </div>
      <p id="error-message" class="error"></p>
      <button type="submit">Confirm</button>
    </form>
  </div>

  <script>
    // Step Navigation
    const step1 = document.getElementById("step1");
    const step2 = document.getElementById("step2");
    const step3 = document.getElementById("step3");

    step1.style.display = "block";
```

```
// Forgot Password Submit
document.getElementById("forgot-form").addEventListener("submit", function (e) {
  e.preventDefault();
  const input = document.getElementById("email").value.trim();
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

  if (!emailRegex.test(input)) {
    alert("Enter a valid email address.");
    return;
  }

  step1.style.display = "none";
  step2.style.display = "block";
});

// OTP Verification
const inputs = document.querySelectorAll(".otp-input");
const verifyBtn = document.getElementById("verify-btn");
const resendBtn = document.getElementById("resend-btn");

function isOtpComplete() {
  return [...inputs].every(input => input.value.trim() !== "");
}

inputs.forEach((input, index) => {
  input.addEventListener("input", (e) => {
    input.value = input.value.replace(/[^0-9]/g, "");
    if (input.value && index < inputs.length - 1) {
      inputs[index + 1].removeAttribute("disabled");
```

```
        inputs[index + 1].focus();
      }
      verifyBtn.disabled = !isOtpComplete();
    });

    input.addEventListener("keydown", (e) => {
      if (e.key === "Backspace" && input.value === "" && index > 0) {
        inputs[index - 1].focus();
      }
    });
});

document.getElementById("otp-form").addEventListener("submit", function (e) {
    e.preventDefault();
    if (isOtpComplete()) {
      alert("OTP Verified Successfully");
      step2.style.display = "none";
      step3.style.display = "block";
    } else {
      alert("Please complete the OTP");
    }
});

resendBtn.addEventListener("click", function (e) {
    e.preventDefault();
    alert("OTP resent!");
    location.reload();
});
```

```
// Toggle Password Visibility
function togglePassword1() {
    const password = document.getElementById("password");
    password.type = password.type === "password" ? "text" : "password";
}

function togglePassword2() {
    const confirm = document.getElementById("confirm-password");
    confirm.type = confirm.type === "password" ? "text" : "password";
}

// Handle Reset Password Submit
document.getElementById("reset-form").addEventListener("submit", async function (e) {
    e.preventDefault();

    const password = document.getElementById("password").value.trim();
    let passwordRegex = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[!@#$%^&*()_+\-=\[\]{}|]).{8,}$/;
    const confirmPassword = document.getElementById("confirm-password").value.trim();
    const email = document.getElementById("email").value.trim();
    if (password.length < 8) {
        errorMessage.textContent = "Password must be at least 8 characters long";
        return;
    }

   if (password.length > 128) {
        errorMessage.textContent = "Password must not exceed 128 characters";
        return;
   }
    if (!passwordRegex.test(password)) {
```

```
    errorMessage.textContent = "Password must include at least one uppercase, one lowercase, one number and one special character";
    return;
  }
  if (password !== confirmPassword) {
    alert("Passwords do not match.");
    return;
  }

  try {
    const response = await fetch("http://localhost:5000/reset-password", {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify({ email, newPassword: password })
    });

    const data = await response.json();
    if (response.ok) {
      alert("Password reset successful.you can now login");
      window.location.href = "login.html"; // Adjust path if needed
    } else {
      alert(data.error || "Failed to reset password");
    }
  } catch (err) {
    console.error("Error:", err);
    alert("Server error. Please try again later.");
  }
```

```
    });
  </script>
</body>
</html>
```

Home.html:

```
<html>
<head>
  <title>Home & Kitchen</title>
  <link rel="stylesheet" href="cat.css">
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
  <script src="temp.js"></script>
</head>
<body>
  <nav class="topnav">
    <div class="logo">QuickMart</div>
    <input type="text" id="loc" placeholder="Enter Pincode" maxlength="6">
    <a class="NavLogo" href="homepg.html">🏠Home</span></a>
    <input type="text" id="search-bar" placeholder="Search QuickMart"
onkeyup="redirectToSearch(event)">
    <div class="user">
    <a class="NavLogo" href="login2.html">Login 👤 </a>
    </div>
    <div class="cart">
    <a class="NavLogo" href="cart2.html">Cart 🛒 <i class="shopping-cart"></i> <span id="cart-
count"></span></a>
    </div>
    <div class="nav3" id="menu-toggle">☰</div>
  </nav>
  <div id="sidebar">
```

```
    <a href="profile.html" class="sidebar-link">My Profile</a>
    <a href="myorders.html" class="sidebar-link">My Orders</a>
    <a href="#" class="sidebar-link" id="logout">Logout</a>
  </div>
  <div class="categories-nav">
    <a href="fruits.html">Fruits & Vegetables</a>
    <a href="home.html" class="active">Home & Kitchen</a>
    <a href="packaged.html">Packaged Foods</a>
    <a href="personalcare.html">Personal Care</a>
  </div>
  <div class="container">
    <div class="sidebar">
      <b>Filter by:</b><br>
      All Products <input type="checkbox" id="all-products" class="category" checked><br>
      Dal & Pulses <input type="checkbox" class="category" value="Dal & Pulses"
name="subcategor"><br>
      Rice & Grains <input type="checkbox" class="category" value="Rice & Grains"
name="subcategory"><br>
      Cookware <input type="checkbox" class="category" value="Cookware" name="subcategory"><br>
      Storage <input type="checkbox" class="category" value="Storage" name="subcategory"><br>
      Cleaning Supplies <input type="checkbox" class="category" value="Cleaning Supplies"
name="subcategory"><br><br>
      <b>Sort by:</b><br>
      <select id="sort-price">
        <option value="asc">Price: Low to High</option>
        <option value="desc">Price: High to Low</option>
      </select>
    </div>

    <div class="products">
```

```
    </div>
  </div>
</body>
</html>
```

Packages foods page : Package.html:

```
<html>
  <head>
    <title>Packaged foods</title>
    <link rel="stylesheet" href="cat.css">
    <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
    <script src="temp.js"></script>
  </head>
  <body>
    <div><nav class="topnav">
      <div class="logo">QuickMart</div>
      <input type="text" id="loc" placeholder="Enter Pincode" maxlength="6">
      <a class="NavLogo" href="homepg.html">🏠 Home</span></a>
      <input type="text" id="search-bar" placeholder="Search QuickMart"
onkeyup="redirectToSearch(event)">
      <div class="user">
        <a class="NavLogo" href="login2.html">Login 👤 </a>
      </div>
      <div class="cart">
        <a class="NavLogo" href="cart2.html">Cart 🛒 <i class="shopping-cart"></i> <span id="cart-
count"></span></a>
      </div>
      <div class="nav3" id="menu-toggle">☰</div>
    </nav>
```

```
    <div id="sidebar">
      <a href="profile.html" class="sidebar-link">My Profile</a>
      <a href="myorders.html" class="sidebar-link">My Orders</a>
      <a href="#" class="sidebar-link" id="logout">Logout</a>
    </div>
    </div>
    <div class="categories-nav">
      <a href="fruits.html">Fruits & Vegetables</a>
      <a href="home.html">Home & Kitchen</a>
      <a href="packaged.html"class="active">Packaged Foods</a>
      <a href="personalcare.html">Personal Care</a>
    </div></div>
    <div class="container">
    <div class="sidebar">
        <b>Filter by:</b><br>
        All Products<input type="checkbox" id="all-products" class="category" checked><br>
        Frozen foods<input type="checkbox" class="category" value="Frozen foods" name="Frozen foods"><br>
        Biscuits and cookies<input type="checkbox" class="category" value="Biscuits and Cookies" name="Biscuits and Cookies"><br>
        Drinks and Juices<input type="checkbox" class="category" value="Drinks and Juices" name="Drinks and Juices"><br>
        Breakfast Cereals<input type="checkbox" class="category" value="Breakfast Cereals" name="Breakfast Cereals"><br>
        Ready foods<input type="checkbox" class="category" value="Ready foods" name="Ready foods"><br><br>
      <b>Sort by:</b><br>
      <select id="sort-price">
        <option value="asc">Price: Low to High</option>
        <option value="desc">Price: High to Low</option>
      </select>
```

```
    </div>
    <div class="products">


    </div>
  </div>
  </body>
</html>
```

Personal care page : personal.html

```
<html>
  <head>
    <title>Personal Care</title>
    <link rel="stylesheet" href="cat.css">
    <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
    <script src="temp.js"></script>
  </head>
  <body>
    <div><nav class="topnav">
      <div class="logo">QuickMart</div>
      <input type="text" id="loc" placeholder="Enter Pincode" maxlength="6">
      <a class="NavLogo" href="homepg.html">🏠 Home</span></a>
      <input type="text" id="search-bar" placeholder="Search QuickMart"
onkeyup="redirectToSearch(event)">
      <div class="user">
      <a class="NavLogo" href="login2.html">Login 👤 </a>
      </div>
      <div class="cart">
      <a class="NavLogo" href="cart2.html">Cart 🛒 <i class="shopping-cart"></i> <span id="cart-
count"></span></a>
      </div>
```

```
    <div class="nav3" id="menu-toggle">☰</div>
  </nav>
  <div id="sidebar">
    <a href="profile.html" class="sidebar-link">My Profile</a>
    <a href="myorders.html" class="sidebar-link">My Orders</a>
    <a href="#" class="sidebar-link" id="logout">Logout</a>
  </div>
  </div>
  <div class="categories-nav">
    <a href="fruits.html">Fruits & Vegetables</a>
    <a href="home.html">Home & Kitchen</a>
    <a href="packaged.html">Packaged Foods</a>
    <a href="personalcare.html" class="active">Personal Care</a>
  </div>
  <div class="container">
  <div class="sidebar">
      <b>Filter by:</b><br>
      All Products<input type="checkbox" id="all-products" class="category" checked><br>
      Body Care<input type="checkbox" class="category" value="Body Care" name="Body Care"><br>
      Face Wash<input type="checkbox" class="category" value="Face Wash" name="Face
Wash"><br>
      Shampoo<input type="checkbox" class="category" value="Shampoo" name="Shampoo"><br>
      Conditioner<input type="checkbox" class="category" value="Conditioner"
name="Conditioner"><br>
      Oral Care<input type="checkbox" class="category" value="Oral Care" name="Oral
Care"><br><br>
      <b>Sort by:</b><br>
      <select id="sort-price">
        <option value="asc">Price: Low to High</option>
        <option value="desc">Price: High to Low</option>
```

```
        </select>
      </div>
      <div class="products">

        </div>
      </div>
    </div>
</body>
</html>
```

PROFILE PAGE : profile.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>View & Edit Profile</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f4f4f4;
      display: flex;
      justify-content: center;
      align-items: center;
      height: 100vh;
    }
    .container {
      background: white;
      padding: 20px;
      border-radius: 5px;
```

```
    width: 400px;
    height: 520px;
    border: 1px solid black;
}
label {
    font-weight: bold;
    display: block;
    margin-top: 10px;
}
input, textarea {
    width: 80%;
    padding: 8px;
    margin-top: 5px;
    border: 1px solid #ccc;
    border-radius: 0px;
    font-size: 14px;
    resize: none;
}
textarea {
    height: 60px;
}
.btn-container {
    display: flex;
    flex-wrap: wrap;
    justify-content: space-between;
    margin-top: 15px;
}
button {
    padding: 10px 15px;
```

```
            border: none;
            cursor: pointer;
            border-radius: 5px;
            margin: 5px 0;
        }
        .edit-btn {
            background-color: green;
            color: white;
        }
        .save-btn {
            background-color: blue;
            color: white;
        }
        .cancel-btn {
            background-color: gray;
            color: white;
        }
        .start-shopping {
            display: inline-block;
            padding: 10px 20px;
            background-color: #007bff;
            color: white;
            text-decoration: none;
            border-radius: 5px;
            margin-top: 10px;
        }
    </style>
</head>
<body>
```

```
<div class="container">
  <h2>My Profile</h2>
  <label>Name</label>
  <input type="text" id="name" disabled>

  <label>Phone Number</label>
  <input type="text" id="phone" disabled>

  <label>Address</label>
  <textarea id="address" disabled></textarea>

  <label>Email</label>
  <input type="text" id="email" disabled>

  <div class="btn-container">
    <button class="edit-btn" onclick="enableEdit()">Edit Profile</button>
    <button class="save-btn" onclick="saveChanges()" style="display: none;">Save Changes</button>
    <button class="cancel-btn" onclick="cancelEdit()" style="display: none;">Cancel</button>
    <a href="homepg.html" class="start-shopping">Back to Home</a>
  </div>
</div>

<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
<script>
  let originalValues = {};

  $(document).ready(function () {
    $.get('/profile', function (data) {
      $('#name').val(data.name);
```

```
      $('#phone').val(data.phone);
      $('#address').val(data.address);
      $('#email').val(data.email);

      originalValues = {
        name: data.name,
        phone: data.phone,
        address: data.address
      };
    }).fail(function (err) {
      alert("Failed to load profile.");
      console.error(err);
    });
});

function enableEdit() {
    $('#name, #phone, #address').prop('disabled', false);
    $('.edit-btn').hide();
    $('.save-btn, .cancel-btn').show();
}

function cancelEdit() {
    $('#name').val(originalValues.name);
    $('#phone').val(originalValues.phone);
    $('#address').val(originalValues.address);
    $('#name, #phone, #address').prop('disabled', true);
    $('.edit-btn').show();
    $('.save-btn, .cancel-btn').hide();
}
```

```
  function saveChanges() {
const name = $('#name').val().trim();
const phone = $('#phone').val().trim();
const address = $('#address').val().trim();

if (!name || !phone || !address) {
  alert("All fields are required.");
  return;
}

if (!/^[A-Z][a-zA-Z ]{1,49}$/.test(name)) {
  alert("Name must start with a capital letter and be 2–50 alphabetic characters.");
  return;
}

if (!/^[6-9]\d{9}$/.test(phone)) {
  alert("Phone must be 10 digits starting with 6, 7, 8 or 9.");
  return;
}

if (!/^.+(,.+)*\s*Chennai-600[01]\d{2}$/.test(address)) {
  alert("Address must end with Chennai-6000xx or 6001xx.");
  return;
}

const updatedData = { name, phone, address };

$.post('/update-profile', updatedData, function (res) {
```

```
      alert("Profile updated successfully!");
      $('#name, #phone, #address').prop('disabled', true);
      $('.edit-btn').show();
      $('.save-btn, .cancel-btn').hide();

      originalValues = { ...updatedData };
    }).fail(function (err) {
      alert("Failed to update profile.");
      console.error(err);
    });
}

  </script>
</body>
</html>
```

CART PAGE : cart2.html

```
<html>
  <head>
    <title>Cart</title>
    <link rel="stylesheet" href="cat.css">
    <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
    <script src="temp.js"></script>
    <style>
      .cart-container {
        display: flex;
        justify-content: space-between;
        width: 550px;
        margin: 0 auto;
```

```
    padding: 20px;
    border: 1px solid #ccc;
    flex-direction:column;
}
.cart-item {
    display: flex;
    justify-content: flex-start;
    padding: 10px;
    border-bottom: 1px solid #ddd;
    height: 150px;
    align-items: center;
    width: 500px;
}

.cart-item img.prod {
    width: 180px;
    height: 150px;
    margin-right: 20px;
}

.cart-item .item-details {
    flex-grow: 1;
}

.item-details p {
    margin: 5px 0;
}

.quantity-control {
```

```
    margin-top: 10px;
    display: flex;
    align-items: center;
}

.quantity-control input {
    width: 40px;
    margin-left: 10px;
}

.delete-btn {
    background: red;
    color: white;
    border: none;
    cursor: pointer;
    padding: 5px;
    margin-top: 10px;
}
img{
    width:170px;
    height:150px;
    border-radius:5px;
}

.payment-details {
    width: 300px;
    padding: 20px;
    background-color: #f8f8f8;
    border: 1px solid #ccc;
```

```
    box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
    display: flex;
    flex-direction: column;
    justify-content: center;
    align-items: center;
}

.payment-details h3 {
    margin-bottom: 15px;
    font-size: 22px;
}

.payment-details p {
    font-size: 18px;
    margin: 10px 0;
}

.place-order-btn {
    background-color: #28a745;
    color: white;
    padding: 12px;
    border: none;
    cursor: pointer;
    text-align: center;
    width: 200px;
    font-size: 18px;
    margin-top: 20px;
    border-radius: 5px;
}
```

```
.total-price {
    font-size: 18px;
    font-weight: bold;
    margin-top: 20px;
}

.empty-cart {
    text-align: center;
    font-size: 18px;
    color: #555;
    margin-top: 20px;
}

.start-shopping {
    display: inline-block;
    padding: 10px 20px;
    background-color: #007bff;
    color: white;
    text-decoration: none;
    border-radius: 5px;
    margin-top: 20px;
}

.categories-nav {
    text-align: center;
    margin-top: 20px;
}
```

```
      .categories-nav a {
        margin: 0 15px;
        text-decoration: none;
        color: #333;
      }

      .categories-nav .active {
        font-weight: bold;
        color: #007bff;
      }



      a.order {
        display: inline-block;
        padding: 10px 20px;
        background-color: #28a745;
        color: white;
        text-decoration: none;
        border-radius: 5px;
        margin-top: 20px;
        width:80%;
        text-align: center;
      }


    </style>
</head>
<body>
    <nav class="topnav">
      <div class="logo">QuickMart</div>
```

```
        <input type="text" id="loc" placeholder="Enter Pincode" maxlength="6">

        <a class="NavLogo" href="homepg.html">🏠 Home</span></a>

        <input type="text" id="search-bar" placeholder="Search QuickMart"
onkeyup="redirectToSearch(event)">

        <div class="user">

          <a class="NavLogo" href="login2.html">Login 👤 </a>

        </div>

        <div class="cart">

          <a class="NavLogo" href="cart2.html">Cart 🛒 <i class="shopping-cart"></i> <span id="cart-
count"></span></a>

        </div>

        <div class="nav3" id="menu-toggle">☰</div>

    </nav>

    <div id="sidebar">

      <a href="profile.html" class="sidebar-link">My Profile</a>

      <a href="myorders.html" class="sidebar-link">My Orders</a>

      <a href="#" class="sidebar-link" id="logout">Logout</a>

    </div>

    <div class="categories-nav">

      <a href="fruits.html">Fruits & Vegetables</a>

      <a href="home.html">Home & Kitchen</a>

      <a href="packaged.html">Packaged Foods</a>

      <a href="personalcare.html">Personal Care</a>

    </div>

    <h2 style="text-align:center;">Your Cart</h2>


     <div class="container">

       <div id="cart-container">

  <div id="cart-items"></div>

  <div id="cart-summary"></div>
```

```
  <button id="continue-shopping" class="start-shopping" onclick="window.location.href='homepg.html'" style="display: none;">Continue Shopping</button>
  <button id="place-order" style="display: none;" onclick="window.location.href='checkout.html'">Place Order</button>

</div>

<script>
$(document).ready(function(){
  loadCart();

  $(document).on("change", ".cart-qty", function() {
    let productId = $(this).data("id");
    let newQty = parseInt($(this).val());
    updateCart(productId, newQty);
  });

  $(document).on("click", ".remove-item", function() {
    let productId = $(this).data("id");
    removeCartItem(productId);
  });
});

function loadCart() {
  $.get("/cart", function(cartItems) {
    if (cartItems.length === 0) {
      $("#cart-items").html("<p> Your cart is empty</p>");
      $("#continue-shopping").show();
      $("#place-order").hide();
```

```
        return;
    }
    $("#continue-shopping").hide();
    $("#place-order").show();

    let totalMRP = 0, totalPayable = 0, cartHTML = "<table>";
    cartHTML +=
"<tr><th>Image</th><th>Item</th><th>Price</th><th>Quantity</th><th>Total</th><th>Action</th></tr>
";
    cartItems.forEach(item => {
        let itemTotal = item.price * item.quantity;
        totalMRP += item.price * item.quantity;
        totalPayable += itemTotal;

        cartHTML += `
          <tr>
            <td><img src="${item.image_url}"></td>
            <td>${item.product_name}</td>
            <td>₹${item.price}</td>
            <td>
              <input type="number" class="cart-qty" data-id="${item.product_id}" value="${item.quantity}"
min="1" max="${item.stock_quantity}">
            </td>
            <td>₹${itemTotal}</td>
            <td><button class="remove-item" data-id="${item.product_id}">Remove</button></td>
          </tr>`;
    });

    cartHTML += "</table>";
    $("#cart-items").html(cartHTML);
```

```
      $("#cart-summary").html(` <p>Total MRP: ₹${totalMRP}</p><p>Delivery Charge: Free</p><p><b>Total: ₹${totalPayable}</b></p>`);
    }).fail(function(err) {
      if (err.status === 401) {
        alert("Please login to view your cart.");
        window.location.href = "login2.html";
      } else {
        alert("Something went wrong loading the cart.");
      }
    });
}

function updateCart(productId, quantity) {
    $.ajax({
      url: "/update-cart",
      type: "POST",
      contentType: "application/json",
      data: JSON.stringify({ product_id: productId, quantity }),
      success: function() {
        loadCart();
      },
      error: function(err) {
        alert(err.responseJSON.error);
      }
    });
}

function removeCartItem(productId) {
    $.ajax({
```

```
      url: "/remove-from-cart",
      type: "DELETE",
      contentType: "application/json",
      data: JSON.stringify({ product_id: productId }),
      success: function() {
        loadCart();
      }
    });
}
</script>
        </div>

  </body>
</html>
```

CHECK OUT : checkout.html

```
<!DOCTYPE html>
<html>
<head>
 <title>Checkout</title>
 <style>
  body {
   font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
   background-color: #f8f9fa;
   margin: 0;
   padding: 20px;
}

.checkout-container {
   max-width: 800px;
```

```
    margin: 0 auto;
    background-color: #ffffff;
    padding: 30px 40px;
    border-radius: 10px;
    box-shadow: 0 4px 12px rgba(0,0,0,0.1);
}

h2 {
    text-align: center;
    margin-bottom: 30px;
    color: #343a40;
}

.cart-item {
    display: flex;
    justify-content: space-between;
    border-bottom: 1px solid #dee2e6;
    padding: 10px 0;
    font-size: 16px;
    color: #495057;
}

.total-amount {
    text-align: right;
    margin-top: 15px;
    font-size: 18px;
    font-weight: bold;
    color: #212529;
}
```

```
.user-info {
  margin-top: 30px;
}

label {
  display: block;
  margin-bottom: 6px;
  font-weight: 500;
  color: #343a40;
}

input[type="text"] {
  width: 100%;
  padding: 10px 12px;
  border: 1px solid #ced4da;
  border-radius: 6px;
  margin-bottom: 20px;
  font-size: 15px;
}

input[type="checkbox"] {
  margin-right: 8px;
  transform: scale(1.2);
}

.checkbox-group {
  display: flex;
  align-items: center;
```

```
    margin-bottom: 25px;
    color: #495057;
}

.checkout-buttons {
    text-align: right;
}

button {
    background-color: #28a745;
    color: white;
    padding: 12px 25px;
    font-size: 16px;
    border: none;
    border-radius: 6px;
    cursor: pointer;
    transition: background-color 0.3s ease;
}

button:hover {
    background-color: #218838;
}

@media screen and (max-width: 600px) {
    .cart-item {
        flex-direction: column;
        align-items: flex-start;
    }
```

```
  .checkout-buttons {
    text-align: center;
  }

  button {
    width: 100%;
    margin-top: 10px;
  }
}

 </style>
</head>
<body>
 <h1>Checkout</h1>
 <div id="cartItems"></div>
 <h3 id="totalAmount">Total: ₹0</h3>

 <h2>Shipping Details</h2>
 <div class="input-group">
  <label>Name: <input type="text" id="name" /></label>
 </div>
 <div class="input-group">
  <label>Address: <input type="text" id="address" /></label>
 </div>
 <div class="input-group">
  <label>Phone: <input type="text" id="phone" maxlength="10"/></label>
 </div>
 <div class="input-group">
  <label><input type="checkbox" id="setDefault" /> Set as default</label>
```

```
</div>
<p id="statusMsg"></p>
<button onclick="placeOrder()">Proceed to Payment</button>

<script>
window.onload = async function () {
 try {
  const res = await fetch("/checkout");
  if (res.status === 401) {
   alert("Please login to access checkout.");
   return;
  }

  const data = await res.json();
  const cartContainer = document.getElementById("cartItems");
  const totalAmount = document.getElementById("totalAmount");

  if (data.cart && data.cart.length > 0) {
   cartContainer.innerHTML = "";
   let total = 0;

   data.cart.forEach(item => {
    const subtotal = item.quantity * item.price;
    total += subtotal;

    const itemDiv = document.createElement("div");
    itemDiv.className = "cart-item";
    itemDiv.innerHTML = `
     <span>${item.product_name}</span>
```

```
        Quantity: ${item.quantity} | Price: ₹${item.price} | Subtotal: ₹${subtotal}
      `;
      cartContainer.appendChild(itemDiv);
    });

    totalAmount.textContent = `Total: ₹${total}`;
  } else {
    cartContainer.innerHTML = "<p>No items in cart.</p>";
    totalAmount.textContent = `Total: ₹0`;
  }

  if (data.customer) {
    document.getElementById("name").value = data.customer.name || "";
    document.getElementById("address").value = data.customer.address || "";
    document.getElementById("phone").value = data.customer.phone || "";
  }

 } catch (error) {
  console.error("Error loading checkout:", error);
  document.getElementById("totalAmount").textContent = `Total: ₹0`;
 }
};

document.getElementById("setDefault").addEventListener("change", async function () {
 const msg = document.getElementById("statusMsg");

 if (this.checked) {
  const name = document.getElementById("name").value.trim();
  const address = document.getElementById("address").value.trim();
```

```
const phone = document.getElementById("phone").value.trim();

if (!name || !address || !phone) {
  msg.textContent = "Please fill in all fields before saving.";
  msg.style.color = "red";
  return;
}

if (!/^[A-Z][a-zA-Z ]{1,49}$/.test(name)) {
  msg.textContent = "Name must be 2–50 alphabetic characters.";
  msg.style.color = "red";
  return;
}

if (!/^[6-9]\d{9}$/.test(phone)) {
  msg.textContent = "Phone must start with 6/7/8/9 and be 10 digits.";
  msg.style.color = "red";
  return;
}

if (!/^.+(,.+)*\s*Chennai-600[01]\d{2}$/.test(address)) {
  msg.textContent = "Address must end with Chennai-6001xx or 6000xx.";
  msg.style.color = "red";
  return;
}

try {
  const response = await fetch("/update-address", {
    method: "POST",
```

```
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ name, address, phone })
    });

    const result = await response.json();
    if (response.ok) {
      msg.textContent = "Saved default address!";
      msg.style.color = "green";
    } else {
      msg.textContent = result.error || "Failed to save.";
      msg.style.color = "red";
    }
  } catch (error) {
    msg.textContent = "Server error.";
    msg.style.color = "red";
  }
 }
});

async function placeOrder() {
 try {
  const name = document.getElementById("name").value.trim();
  const address = document.getElementById("address").value.trim();
  const phone = document.getElementById("phone").value.trim();
  const setDefault = document.getElementById("setDefault").checked;

  if (!name || !address || !phone) {
    alert("All fields are required.");
    return;
```

```
}

if (!/^[A-Z][a-zA-Z ]{1,49}$/.test(name)) {
  alert("Name must be 2–50 alphabetic characters.");
  return;
}

if (!/^[6-9]\d{9}$/.test(phone)) {
  alert("Phone must start with 6/7/8/9 and be 10 digits.");
  return;
}

if (!/^.+(,.+)*\s*Chennai-600[01]\d{2}$/.test(address)) {
  alert("Address must end with Chennai-6001xx or 6000xx.");
  return;
}

const res = await fetch("/place-order", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({ address, phone, setDefault })
});

const data = await res.json();
if (res.ok) {
  alert("Proceeding to payment");
  window.location.href = `payment.html?order_id=${data.order_id}`;
```

```
    } else {
      alert(data.error || "Order failed.");
    }
  } catch (err) {
    alert("Something went wrong.");
  }
 }
</script>
</body>
</html>
```

PAYMENT PAGE : payment.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Payment Page</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background: #f4f4f4;
    }
    .container {
      max-width: 600px;
      margin: 50px auto;
      background: #fff;
      padding: 30px 40px;
      border-radius: 10px;
      box-shadow: 0 2px 8px rgba(0,0,0,0.1);
    }
    h2 {
```

```
    text-align: center;
    margin-bottom: 25px;
}
label {
    font-weight: bold;
    margin-top: 15px;
    display: block;
}
input, select {
    width: 100%;
    padding: 10px;
    margin-top: 6px;
    border: 1px solid #ccc;
    border-radius: 6px;
}
.message {
    color: #d62828;
    font-size: 14px;
    margin-top: 6px;
}
button {
    width: 100%;
    margin-top: 30px;
    padding: 12px;
    font-size: 16px;
    background: #28a745;
    color: white;
    border: none;
    border-radius: 6px;
```

```
      cursor: pointer;
    }
    button:hover {
      background: #218838;
    }
    #card-type {
      margin-top: 5px;
      color: #555;
    }
  </style>
</head>
<body>
<div class="container">
  <h2>Complete Your Payment</h2>
  <p><strong>Order ID:</strong> <span id="order-id">Loading...</span></p>
  <p><strong>Total Amount:</strong> Rs.<span id="total-amount">0</span></p>

  <label for="payment-method">Payment Method:</label>
  <select id="payment-method">
    <option value="">-- Select Payment Method --</option>
    <option value="Card">Card</option>
    <option value="UPI">UPI</option>
    <option value="Netbanking">Netbanking</option>
    <option value="Cash on Delivery">Cash on Delivery</option>
  </select>

  <div id="card-details" style="display:none;">
    <label>Card Number:</label>
    <input type="text" id="card_number" maxlength="19" placeholder="XXXX XXXX XXXX XXXX">
```

```
    <div id="card-type"></div>
    <div class="message" id="card-number-msg"></div>

    <label>Name on Card:</label>
    <input type="text" id="card_name" placeholder="Full Name">
    <div class="message" id="card-name-msg"></div>

    <label>CVV:</label>
    <input type="password" id="cvv" maxlength="4" placeholder="CVV">
    <div class="message" id="cvv-msg"></div>

    <label>Expiry (MM/YY):</label>
    <input type="text" id="expiry" placeholder="MM/YY" maxlength="5">
    <div class="message" id="expiry-msg"></div>
  </div>

  <div id="upi-details" style="display:none;">
    <label>Enter UPI ID:</label>
    <input type="text" id="upi_id" placeholder="example@upi">
    <div class="message" id="upi-msg"></div>
  </div>

  <div id="netbanking-details" style="display:none;">
    <label>Select Bank:</label>
    <select id="bank">
      <option value="">-- Select Bank --</option>
      <option>HDFC</option>
      <option>SBI</option>
      <option>ICICI</option>
```

```
        <option>Indian Bank</option>
        <option>Kotak Mahindra Bank</option>
        <option>Axis Bank</option>
        <option>Others</option>
      </select>
    </div>

    <button id="pay-btn">Submit</button>
    <div class="message" id="message"></div>
  </div>

  <script>
    let orderId = null, totalAmount = 0;

    window.onload = async () => {
      const res = await fetch("/get-latest-order");
      const data = await res.json();
      orderId = data.order_id;
      totalAmount = data.total;
      document.getElementById("order-id").textContent = orderId;
      document.getElementById("total-amount").textContent = totalAmount;
    };

    document.getElementById("payment-method").addEventListener("change", () => {
      const method = document.getElementById("payment-method").value;
      document.getElementById("card-details").style.display = method === "Card" ? "block" : "none";
      document.getElementById("upi-details").style.display = method === "UPI" ? "block" : "none";
      document.getElementById("netbanking-details").style.display = method === "Netbanking" ? "block" :
  "none";
```

```
    document.getElementById("message").textContent = "";
  });

  document.getElementById("card_number").addEventListener("input", function () {
    const number = this.value.replace(/\s+/g, '');
    const ct = document.getElementById("card-type");
    let type = '';
    if (/^4\d{12}(\d{3})?$/.test(number)) type = "Visa";
    else if (/^5[1-5]\d{14}$/.test(number) || /^2(2[2-9]|[3-6]|7[01])\d{12}$/.test(number)) type =
"MasterCard";
    else if (/^6\d{15}$/.test(number)) type = "RuPay";
    else if (/^3[47]\d{13}$/.test(number)) type = "AmEx";
    else if (/^3(?:6|8)\d{12,14}$/.test(number)) type = "Diners Club";
    else if (/^(50|5[6-9]|6\d)\d{10,17}$/.test(number)) type = "Maestro";
    ct.textContent = type ? "Card Type: " + type : "";
  });

  document.getElementById("pay-btn").addEventListener("click", async () => {
    const method = document.getElementById("payment-method").value;
    if (!method) {
      document.getElementById("message").textContent = "Please select a payment method.";
      return;
    }

    let valid = true;

    // Clear all messages
    ["card-number-msg", "card-name-msg", "cvv-msg", "expiry-msg", "upi-msg", "message"].forEach(id
=> {
      document.getElementById(id).textContent = "";
```

```
});

if (method === "Card") {
  const number = document.getElementById("card_number").value.trim().replace(/\s+/g, '');
  const name = document.getElementById("card_name").value.trim();
  const cvv = document.getElementById("cvv").value.trim();
  const expiry = document.getElementById("expiry").value.trim();

  if (!/^\d{13,19}$/.test(number)) {
    document.getElementById("card-number-msg").textContent = "Invalid card number.";
    valid = false;
  }
  if (!/^[A-Za-z ]{2,50}$/.test(name)) {
    document.getElementById("card-name-msg").textContent = "Invalid name.";
    valid = false;
  }
  if (!/^\d{3,4}$/.test(cvv)) {
    document.getElementById("cvv-msg").textContent = "Invalid CVV.";
    valid = false;
  }

  if (!/^\d{2}\/\d{2}$/.test(expiry)) {
    document.getElementById("expiry-msg").textContent = "Invalid expiry format (MM/YY).";
    valid = false;
  } else {
    const [mm, yy] = expiry.split("/").map(x => parseInt(x));
    const now = new Date();
    const expMonth = mm;
    const expYear = 2000 + yy;
```

```
      const currentYear = now.getFullYear();
      const currentMonth = now.getMonth() + 1;
      if (expYear < currentYear || (expYear === currentYear && expMonth <= currentMonth)) {
        document.getElementById("expiry-msg").textContent = "Card expired.";
        valid = false;
      }
    }
}

if (method === "UPI") {
    const upi = document.getElementById("upi_id").value.trim();
    if (!/^[\w.-]+@[\w]+$/.test(upi)) {
      document.getElementById("upi-msg").textContent = "Invalid UPI ID.";
      valid = false;
    }
}

if (method === "Netbanking") {
    const bank = document.getElementById("bank").value;
    if (!bank) {
      document.getElementById("message").textContent = "Please select your bank.";
      valid = false;
    }
}

if (!valid) return;

try {
    const res = await fetch("/payment", {
```

```
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
          order_id: orderId,
          payment_method: method,
          amount_paid: totalAmount,
          payment_status: "Success"
        })
      });
      const data = await res.json();
      alert("Order confirmed via " + method);
      window.location.href = "homepg.html";
    } catch (err) {
      document.getElementById("message").textContent = "Payment failed.";
    }
  });
</script>
</body>
</html>
```

MY ORDERS : myorder.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My Orders</title>
  <style>
    body {
      font-family: Arial, sans-serif;
```

```
    background: #f8f8f8;
    margin: 0;
    padding: 20px;
}

h2 {
    text-align: center;
    margin-bottom: 30px;
}

.order-item {
    background: #fff;
    border: 1px solid #ddd;
    padding: 20px;
    margin-bottom: 20px;
    border-radius: 8px;
}

.product-list {
    display: flex;
    align-items: center;
    margin-top: 10px;
}

.product-image {
    width: 80px;
    height: 80px;
    object-fit: cover;
    border-radius: 6px;
```

```
    margin-right: 15px;
}

.product-details p {
    margin: 4px 0;
}

.payment-type, .status {
    margin-top: 10px;
}

.status.processing {
    color: orange;
}

.status.shipped {
    color: green;
}

.cancel-btn {
    margin-top: 15px;
    padding: 8px 16px;
    background: #e74c3c;
    color: white;
    border: none;
    border-radius: 4px;
    cursor: pointer;
}
```

```
      .cancel-btn:disabled {
        background: #ccc;
        cursor: not-allowed;
      }
    </style>
</head>
<body>

<h2>My Orders</h2>
<div id="order-list"></div>

<script>
document.addEventListener("DOMContentLoaded", () => {
    fetch("/myorders")
      .then(response => response.json())
      .then(data => {
        if (!data.success) {
          alert(data.message);
          return;
        }
        renderOrders(data.orders);
      })
      .catch(err => console.error("Error fetching orders:", err));
});

function renderOrders(orders) {
    const container = document.getElementById("order-list");
    container.innerHTML = "";
```

```
  orders.forEach(order => {
    const orderDiv = document.createElement("div");
    orderDiv.className = "order-item";
    orderDiv.id = `order-${order.order_id}`;

    const statusClass = getStatusClass(order.status);
    const disableCancel = (order.status === "shipped" || order.status === "delivered");

    let html = `
      <p><strong>Order ID:</strong> #${order.order_id}</p>
    `;

    order.items.forEach(item => {
      html += `
        <div class="product-list">
          <div class="product-details">
            <p><strong>${item.product_name}</strong></p>
            <p>Price: ₹${item.price} × ${item.quantity}</p>
          </div>
        </div>
      `;
    });

    html += `
      <p><strong>Total:</strong> ₹${order.total}</p>
      <p class="status ${statusClass}"><strong>Status:</strong> ${order.status}</p>
      <button class="cancel-btn" onclick="cancelOrder('${order.order_id}', this)" ${disableCancel ? "disabled" : ""}>Cancel Order</button>
    `;
```

```
    orderDiv.innerHTML = html;
    container.appendChild(orderDiv);
  });
}

function getStatusClass(status) {
  if (status === "processing" || status === "placed") return "processing";
  if (status === "shipped" || status === "delivered") return "shipped";
  return "";
}

function cancelOrder(orderId, button) {
  const confirmCancel = confirm("Are you sure you want to cancel this order?");
  if (!confirmCancel) return;

  fetch("/cancel-order", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ order_id: orderId })
  })
  .then(res => res.json())
  .then(data => {
    alert(data.message);
    if (data.success) {
      document.getElementById(`order-${orderId}`).remove();
    }
  })
  .catch(err => console.error("Cancel failed:", err));
```

```
}
</script>


</body>
</html>
```

CSS FILE FOR CATEGORIES : cat.css

```
.products{
   flex-grow:1;
   display:grid;
   grid-template-columns:repeat(auto-fill, minmax(200px, 1fr));
   gap:20px;
}
.product-box{
   background:white;
   padding:10px;
   border-radius:5px;
   box-shadow:2px 2px 10px rgba(0, 0, 0, 0.1);
   width:200px;
   text-align:center;
}
.NavLogo{
   width:100px;
   background-color: white;
   color:black;
   text-decoration: none;
}
button.atc{
   color:white;
   background-color:rgb(6, 147, 65);
```

```
    height:30px;
    width:100px;
    border:none;
    cursor:pointer;
}
button.atc:active{
    transform:scale(0.95);
}
button.atc:hover{
    background-color:rgb(159, 217, 183);
}
.product-box img{
    width:170px;
    height:150px;
    border-radius:5px;
}
.disc{
    font-size:15px;
    background-color:rgb(127, 255, 148);
    width:100px;
}
s{
    color:rgb(174, 171, 171);
    font-size:17px;
}
.sidebar{
    text-align:center;
    width:200px;
    padding:20px;
```

```
    background-color: white;
    box-shadow: 2px 2px 10px rgba(0, 0, 0, 0.1);
}

.nav-item{
    display:block;
    width:100%;
    margin:5px 0;
    padding: 8px;
    color:black;
    cursor: pointer;
}

.nav-item.active{
    background-color:rgb(6, 147, 65);
    color:white;
}

.container{
    display:flex;
    align-items:flex-start;
    gap:20px;
    padding:20px;
}

.logo{
    font-size:24px;
    font-weight:bold;
}
```

```
.topnav{
   display:flex;
   justify-content:space-between;
   background-color:#28a745;
   padding:10px 20px;
   color: white;
   align-items:center;
}
.topnav input{
   padding:8px;
   width:250px;
   border:none;
   border-radius:5px;
}
.category{
   accent-color:rgb(6, 147, 65);
}
.no-results{
   text-align:center;
   font-size:18px;
   color:rgb(6, 147, 65);
   font-weight:bold;
   margin-top:20px;
}
body{
   margin:0;
   padding:0;
}
```

```
.categories-nav{
   background-color:#f8f8f8;
   padding:10px;
   text-align:center;
   border-bottom:2px solid #ddd;
}
.categories-nav a{
   text-decoration:none;
   color:#333;
   font-weight:bold;
   padding:10px 15px;
   margin:0 10px;
   display:inline-block;
   transition:0.3s;
}

.categories-nav a.active{
   color:green;
}
#navbar-container{
   width:100%;
   min-height:50px;
   border:none;
   background:white;
}
#navbar-container *{
   display:block !important;
}
#search-bar{
```

```
    width:400px;
}
.NavLogo:active{
    text-decoration:none;
}
.banner {
    width: 100vw;
    height: 300px;
    overflow: hidden;
    position: relative;
    cursor: pointer;
    transition: 0.3s;
    opacity: 0.9;
    display: flex;
    align-items: center;
    justify-content: center;
}

.banner img {
    width: 100%;
    height: 100%;
    object-fit: fill;
}

.category-container {
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: center;
```

```
    gap: 20px;
    flex-wrap: wrap;
    padding: 20px;
}

.category2 {
    width: 100%;
    height: 300px;
    border-radius: 10px;
    overflow: hidden;
    position: relative;
    cursor: pointer;
}

.category img {
    width: 100%;
    height: 100%;
    object-fit: fill;
    transition: 0.3s;
}

.category:hover img {
    opacity: 0.8;
}

.footer {
    background: #333;
    color: white;
    text-align: center;
```

```
    padding: 20px;
    margin-top: 20px;
}

.footer a {
    color: white;
    text-decoration: none;
    margin: 0 15px;
    font-weight: bold;
}

.footer a:hover {
    text-decoration: underline;
}
body {
    font-family: Arial, sans-serif;
}

#sidebar {
    position: fixed;
    top: 50px;
    right: 0;
    width: 200px;
    background: white;
    box-shadow: -2px 0 5px rgba(0, 0, 0, 0.2);
    padding: 15px;
    display: none;
    z-index: 1000;
}
```

```
.sidebar-link {
   display: block;
   padding: 10px;
   color: black;
   text-decoration: none;
}

.sidebar-link:hover {
   background: lightgray;
}

.nav3{
 cursor:pointer;
}
```

VALIDITY CHECK JS FILE : temp.js

```
$(document).ready(() => {
   function fetchProducts() {
      let pageCategory = "";
      if (window.location.pathname.includes("fruits.html")) pageCategory = "CAT001";
      else if (window.location.pathname.includes("home.html")) pageCategory = "CAT002";
      else if (window.location.pathname.includes("packaged.html")) pageCategory = "CAT003";
      else if (window.location.pathname.includes("personalcare.html")) pageCategory = "CAT004";

      let subcategoryFilter = [];

      if ($("#all-products").prop("checked")) {
         subcategoryFilter = [];
```

```
} else {
   $(".category:checked").each(function () {
      let subcat = $(this).val().trim();
      if (subcat) subcategoryFilter.push(encodeURIComponent(subcat));
   });
}

let queryParams = [`category=${pageCategory}`];

if (subcategoryFilter.length > 0) {
   queryParams.push(`subcategories=${subcategoryFilter.join(",")}`);
}

let sortOrder = $("#sort-price").val();
if (sortOrder) {
   queryParams.push(`sort=${sortOrder}`);
}

let queryString = queryParams.length > 0 ? "?" + queryParams.join("&") : "";

console.log("Final Query String Sent to Server:", queryString);

$.get(`/products${queryString}`, (products) => {
   $(".products").empty();

   if (products.length === 0) {
      $(".products").html("<p>No products found</p>");
      return;
   }
```

```
      products.forEach((prod) => {
        let discount = Math.round(((prod.price - prod.current_price) / prod.price) * 100);

        let productHTML = `
        <div class="product-box" data-product-id="${prod._id}">
          <img src="${prod.image_url}" class="prod">
          <p>${prod.name}<br>${prod.description}<br>
            <s>₹${prod.price}</s>  
            <span class="disc">${discount}% OFF</span><br>
            ₹${prod.current_price}
          </p>
          <button class="atc">Add to cart</button>
        </div>`;

        $(".products").append(productHTML);
      });
    }).fail((err) => {
      console.error("Error fetching products:", err);
    });
}

$("#all-products").change(function () {
    if ($(this).prop("checked")) {
      $(".category").prop("checked", false);
    }
    fetchProducts();
});
```

```
$(".category").change(function () {
    if ($(".category:checked").length > 0) {
        $("#all-products").prop("checked", false);
    } else {
        $("#all-products").prop("checked", true);
    }
    fetchProducts();
});

$("#sort-price").change(fetchProducts);

fetchProducts();

$(document).on("click", ".atc", function () {
    const productBox = $(this).closest(".product-box");

    const productData = {
        product_id: productBox.data("product-id"),
        product_name: productBox.find("p").contents().get(0).nodeValue.trim(),
        current_price: parseFloat(productBox.find("span:last").text().replace("₹", "")),
        quantity: 1
    };

    $.post("/add-to-cart", productData, function (response) {
        alert(response.message);
    }).fail(function (err) {
        alert(err.responseJSON?.error || "Error adding to cart");
    });
});
```

```
$("#search-bar").keypress(function (event) {
    if (event.key === "Enter") {
        let query = $(this).val().trim();
        if (query) {
            window.location.href = `searchbar.html?q=${encodeURIComponent(query)}`;
        }
    }
});
$(".nav3, #sidebar").hover(
    function() {
        $("#sidebar").fadeIn();
    },
    function() {
        $("#sidebar").fadeOut();
    }
);

$("#logout").click(function (event) {
    event.preventDefault();

    $.post("/logout", function (response) {
        alert(response.message);
        window.location.href = "homepg.html";
    }).fail(function (err) {
        alert("Logout failed. Please try again.");
    });
});

$("#loc").keypress(function (e) {
```

```
    if (e.key === "Enter") {
      let pincode = $(this).val().trim();
      const chennaiPattern = /^600[01]\d{2}$/;

      if (chennaiPattern.test(pincode)) {
        alert("Location accessible");
      } else {
        alert("Location not accessible");
      }
    }
  });

});
```

BACKEND FILE : server1.js

```
const mongoose = require('mongoose');
const express = require('express');
const bodyParser = require('body-parser');
const cors = require("cors");
const PORT = 5000;
const app = express();
const schema = mongoose.Schema;

app.use(express.static(__dirname));
app.use(bodyParser.urlencoded({ extended: true }));
app.use(bodyParser.json());

const url = "mongodb://127.0.0.1:27017/Groceries";
const session = require("express-session");
```

```
app.use(
   session({
     secret: "your_secret_key",
     resave: false,
     saveUninitialized: false,
     cookie: {
       secure: false,
       maxAge: 24 * 60 * 60 * 1000
     }
   })
 );



mongoose.connect(url)
  .then(() => console.log("Connected with MongoDB"))
  .catch(error => console.log("Error in DB Connection:", error));

const ProductSchema = new schema({
  _id: String,
  name: String,
  category_id: String,
  subcat: String,
  price: Number,
  current_price: Number,
  stock_quantity: Number,
  image_url: String
});
```

```
const ProductModel = mongoose.model('Products', ProductSchema, 'Products');


const CartSchema = new schema({
 cart_id: String,
 customer_id: String,
 product_id: String,
 product_name: String,
 price: Number,
 quantity: Number
});

CartSchema.index({ customer_id: 1, product_id: 1 }, { unique: true });

const CartModel = mongoose.model("Cart", CartSchema, "Carts");

app.get('/products', async (req, res) => {
  try {
    let filter = { stock_quantity: { $gt: 0 } };

    if (req.query.search) {
      filter.name = { $regex: req.query.search, $options: "i" };
    }
    if (req.query.category) {
      filter.category_id = req.query.category;
    }
    if (req.query.subcategories) {
      let subcategories = req.query.subcategories.split(",").map(decodeURIComponent);
      filter.subcat = { $in: subcategories };
    }
```

```
    let sortOption = {};
    if (req.query.sort === "asc") {
      sortOption.current_price = 1;
    } else if (req.query.sort === "desc") {
      sortOption.current_price = -1;
    }

    const products = await ProductModel.find(filter).sort(sortOption);
    res.json(products);
  } catch (error) {
    console.error("Error fetching products:", error);
    res.status(500).json({ error: "Internal Server Error" });
  }
});

app.post('/add-to-cart', async (req, res) => {
  try {
    let { product_id, product_name, quantity } = req.body;
    if (!product_id || typeof product_id !== "string") {
      return res.status(400).json({ error: "Invalid product ID" });
    }

    const customer_id = req.session.customer_id;
    if (!customer_id) {
      return res.status(401).json({ error: "User not logged in" });
    }
    let existingCart = await CartModel.findOne({ customer_id });
    const cart_id = existingCart ? existingCart.cart_id : `CART_${customer_id}`;
```

```
    const existingItem = await CartModel.findOne({ customer_id, product_id });

    if (existingItem) {
      return res.status(400).json({ error: "Item is already in cart" });
    }

    const product = await ProductModel.findOne({ _id: product_id });

    if (!product) {
      return res.status(400).json({ error: "Product not found" });
    }

    if (quantity > product.stock_quantity) {
      return res.status(400).json({ error: `Cannot add more than available stock quantity of ${product.stock_quantity}` });
    }

    const cartItem = new CartModel({
      cart_id,
      customer_id,
      product_id,
      product_name,
      price: product.current_price,
      quantity: quantity || 1
    });

    await cartItem.save();
    res.json({ message: "Item added to cart" });
```

```
  } catch (error) {
    res.status(500).json({ error: "Server error" });
  }
});

app.get('/cart', async (req, res) => {
  try {
    const customer_id = req.session.customer_id;
    if (!customer_id) {
      return res.status(401).json({ error: "User not logged in" });
    }
    const cartItems = await CartModel.find({ customer_id });

    let updatedCart = await Promise.all(cartItems.map(async (item) => {
      let product = await ProductModel.findOne({ _id: item.product_id });
      return { ...item._doc, image_url: product ? product.image_url : "",  stock_quantity: product ? product.stock_quantity : 0 };
    }));

    res.json(updatedCart);
  } catch (error) {
    console.error("Error fetching cart:", error);
    res.status(500).json({ error: "Error fetching cart" });
  }
});

app.post('/update-cart', async (req, res) => {
  try {
```

```
    const { product_id, quantity } = req.body;
    const customer_id = req.session.customer_id;
    if (!customer_id) {
      return res.status(401).json({ error: "User not logged in" });
    }

    let product = await ProductModel.findOne({ _id: product_id });

    if (!product || quantity > product.stock_quantity) {
      return res.status(400).json({ error: `Cannot exceed stock quantity of ${product ? product.stock_quantity : 0}` });
    }

    await CartModel.updateOne({ customer_id, product_id }, { $set: { quantity } });
    res.json({ message: "Cart updated successfully" });
  } catch (error) {
    res.status(500).json({ error: "Error updating cart" });
  }
});

app.delete('/remove-from-cart', async (req, res) => {
  try {
    const { product_id } = req.body;
    const customer_id = req.session.customer_id;
    if (!customer_id) {
      return res.status(401).json({ error: "User not logged in" });
    }

    await CartModel.deleteOne({ customer_id, product_id });
```

```
    res.json({ message: "Item removed from cart" });
  } catch (error) {
    res.status(500).json({ error: "Error removing item" });
  }
});

const userSchema = new mongoose.Schema({
  customer_id: { type: String, unique: true, index: true },
  name: { type: String, required: true },
  email: { type: String, unique: true, required: true },
  password: { type: String, required: true },
  address: { type: String, default: "" },
  phone: { type: String, default: "" }
  }, { versionKey: false });

userSchema.pre("save", async function (next) {
  if (!this.customer_id) {
    let count = await mongoose.models.User.countDocuments();
    this.customer_id = `CUST${String(count + 1).padStart(3, '0')}`;
  }
  next();
});

const User = mongoose.model("User", userSchema, "users");

app.post("/signup", async (req, res) => {
  const { name, email, password } = req.body;

  if (!name || !email || !password) {
```

```
    return res.status(400).json({ error: "All fields are required" });
  }

  try {
    const existingUser = await User.findOne({ email });
    if (existingUser) {
      return res.status(400).json({ error: "Email already registered" });
    }

    const newUser = new User({ name, email, password });
    await newUser.save();
    res.json({ message: "User registered successfully", customer_id: newUser.customer_id });
  } catch (err) {
    console.error("Signup Error:", err);
    res.status(500).json({ error: "Internal Server Error" });
  }
});
app.post('/login2', async (req, res) => {
  if (req.session.customer_id) {
    console.log("User already logged in:", req.session.customer_id);
    return res.status(400).json({ message: "User already logged in" });
   }
  const { email, password } = req.body;

  try {
    const user = await User.findOne({ email }).lean();

    if (!user) {
      return res.status(400).json({ message: 'Customer not found' });
```

```
    }

    if (password !== user.password) {
      return res.status(400).json({ message: 'Invalid password' });
    }
    req.session.customer_id = user.customer_id;
    const existingCart = await CartModel.find({ customer_id: user.customer_id });
    console.log("Session after login:", req.session);
    req.session.save(err => {
      if (err) {
        console.log("Error saving session:", err);
        return res.status(500).json({ message: "Error saving session" });
      }
      console.log("Login successful, session saved:", req.session.customer_id);
      res.status(200).json({
        message: 'Login successful',
        customer_id: user.customer_id,
        cart: existingCart
      });

    });

  } catch (error) {
    console.log("Server error during login:", error);
    res.status(500).json({ message: 'Server error' });
  }

});
```

```
app.get("/session", (req, res) => {
  console.log("Checking session:", req.session);
  console.log("Session customer_id:", req.session.customer_id);
  if (req.session.customer_id) {
    console.log("Session found:", req.session.customer_id);
    return res.json({ loggedIn: true, customer_id: req.session.customer_id });
  }
  console.log("No active session");
  res.json({ loggedIn: false });
});

app.post("/logout", (req, res) => {
  req.session.destroy((err) => {
    if (err) {
      return res.status(500).json({ error: "Logout failed" });
    }
    res.clearCookie("connect.sid");
    res.json({ message: "Logged out successfully" });
  });
});

app.get("/checkout", async (req, res) => {
  const customer_id = req.session.customer_id;
  if (!customer_id) {
    return res.status(401).json({ error: "User not logged in" });
  }

  try {
    const cartItems = await CartModel.find({ customer_id });
```

```
    const total = cartItems.reduce((acc, item) => acc + (item.price * item.quantity), 0);
    const user = await User.findOne({ customer_id }).lean();

    res.json({
      cart: cartItems,
      total,
      customer: {
        name: user.name,
        address: user.address,
        phone: user.phone
      }
    });
  } catch (error) {
    console.error("Error in /checkout route:", error);
    res.status(500).json({ error: "Internal server error" });
  }

});

app.post("/update-address", async (req, res) => {
  const customer_id = req.session.customer_id;
  if (!customer_id) {
    return res.status(401).json({ error: "User not logged in" });
  }

  const { name, address, phone } = req.body;
  try {
    await User.updateOne({ customer_id }, { $set: { name, address, phone } });
    res.json({ message: "Details saved as default" });
```

```
  } catch (err) {
    console.error("Error updating user info:", err);
    res.status(500).json({ error: "Failed to update" });
  }
});

const OrderSchema = new mongoose.Schema({
  order_id: String,
  customer_id: String,
  name: String,
  address: String,
  phone: String,
  items: [
   {
    product_id: String,
    product_name: String,
    quantity: Number,
    price: Number
   }
  ],
  total: Number,
  payment_status: {
    type: String,
    default: "pending"
   },
  placed_at: {
   type: Date,
   default: Date.now
  },
```

```
  status: {
    type: String,
    default: "placed"
  }
}, { versionKey: false });

const OrderModel = mongoose.model("Order", OrderSchema, "orders");


app.post("/place-order", async (req, res) => {
  const customer_id = req.session.customer_id;

  if (!customer_id) {
    return res.status(401).json({ error: "User not logged in" });
  }

  try {
    const { address, phone, setDefault } = req.body;
    console.log("Received body:", req.body);

    const user = await User.findOne({ customer_id });
    if (!user) {
      return res.status(404).json({ error: "User not found" });
    }

    const cartItems = await CartModel.find({ customer_id });

    if (!cartItems.length) {
      return res.status(400).json({ error: "Cart is empty" });
```

```
}

const total = cartItems.reduce(
 (sum, item) => sum + item.quantity * item.price,
 0
);

await OrderModel.deleteMany({ customer_id, payment_status: "pending" });

const orderCount = await OrderModel.countDocuments();
const order_id = `ORDER${String(orderCount + 1).padStart(4, "0")}`;

const order = new OrderModel({
 order_id,
 customer_id,
 name: user.name,
 address,
 phone,
 items: cartItems.map((item) => ({
  product_id: item.product_id,
  product_name: item.product_name,
  quantity: item.quantity,
  price: item.price,
 })),
 total,
 payment_status: 'pending',
 placed_at: new Date()
});
```

```
    console.log("Saving order:", order);

    const result = await order.save();
    console.log("Order saved:", result);

    if (setDefault) {
      await User.updateOne({ customer_id }, { $set: { address, phone } });
    }

    res.json({ message: "Proceeding to payment", order_id });
  } catch (err) {
    console.error("Error placing order:", err);
    res.status(500).json({ error: "Failed to place order" });
  }
});



app.get('/profile', async (req, res) => {
    const customer_id = req.session.customer_id;
    if (!customer_id) {
        return res.status(401).json({ error: "User not logged in" });
    }

    try {
        const user = await User.findOne(
            { customer_id },
            { _id: 0, name: 1, email: 1, address: 1, phone: 1 }
        );
```

```
        if (!user) {
            return res.status(404).json({ error: "User not found" });
        }

        res.json(user);
    } catch (error) {
        console.error("Error fetching user profile:", error);
        res.status(500).json({ error: "Server error" });
    }
});

app.post('/update-profile', async (req, res) => {
  const customer_id = req.session.customer_id;
  if (!customer_id) {
    return res.status(401).json({ error: "User not logged in" });
  }

  const { name, phone, address } = req.body;

  try {
    const updatedUser = await User.findOneAndUpdate(
      { customer_id },
      { name, phone, address },
      { new: true }
    );

    if (!updatedUser) {
      return res.status(404).json({ error: "User not found" });
    }
```

```
      res.json({ message: "Profile updated successfully", user: updatedUser });
    } catch (error) {
      console.error("Error updating user profile:", error);
      res.status(500).json({ error: "Server error" });
    }
});

const paymentSchema = new mongoose.Schema({
    order_id: String,
    customer_id: String,
    payment_method: String,
    amount_paid: Number,
    payment_status: String,
    payment_time: { type: Date, default: Date.now }
  });
  app.get("/get-latest-order", async (req, res) => {
    const customer_id = req.session.customer_id;
    if (!customer_id) return res.status(401).json({ error: "Not logged in" });

    try {
      const order = await OrderModel.findOne({ customer_id })
        .sort({ placed_at: -1 })
        .limit(1);
      if (!order) return res.status(404).json({ error: "Order not found" });

      res.json({ order_id: order.order_id, total: order.total });
    } catch (err) {
      console.error("Error fetching latest order:", err);
```

```
    res.status(500).json({ error: "Server error" });
  }
 });

const PaymentModel = mongoose.model('Payment', paymentSchema);
app.post("/payment", async (req, res) => {
  const customer_id = req.session.customer_id;

  if (!customer_id) {
    return res.status(401).json({ error: "User not logged in" });
  }

  const { order_id, payment_method, amount_paid, payment_status } = req.body;

  try {
    const order = await OrderModel.findOne({ order_id });

    if (!order) {
      return res.status(404).json({ error: "Order not found" });
    }

    if (payment_status === "Success") {
      const payment = new PaymentModel({
        order_id,
        payment_method,
        amount_paid,
        payment_status,
        payment_time: new Date()
      });
```

```
        await OrderModel.updateOne({ order_id }, { $set: { payment_status: "paid" } });
        await payment.save();

        await CartModel.deleteMany({ customer_id });

        for (const item of order.items) {
          const result = await ProductModel.updateOne(
            { _id: item.product_id },
            { $inc: { stock_quantity: -item.quantity } }
          );
          console.log(`Updated ${item.product_id} =>`, result);
        }

        res.json({ message: "Payment successful, order confirmed!" });
      } else {
        await OrderModel.deleteOne({ order_id });

        res.json({ message: "Payment failed or cancelled. Order removed." });
      }

    } catch (error) {
      console.error("Error in /payment:", error);
      res.status(500).json({ error: "Payment processing failed" });
    }
});

app.get("/myorders", async (req, res) => {
  if (!req.session.customer_id) {
    return res.json({ success: false, message: "Please log in to view your order history." });
```

```
  }

  try {
    const customer_id = req.session.customer_id;
    const orders = await OrderModel.find({ customer_id }).sort({ placed_at: -1 });
    res.json({ success: true, orders });
  } catch (err) {
    console.error("Error fetching orders:", err);
    res.status(500).json({ success: false, message: "Server error fetching orders." });
  }
});

app.post("/cancel-order", async (req, res) => {
  try {
    const { order_id } = req.body;

    const order = await OrderModel.findOne({ order_id });

    if (!order) {
      return res.json({ success: false, message: "Order not found" });
    }

    for (const item of order.items) {
      await ProductModel.updateOne(
        { _id: item.product_id },
        { $inc: { stock_quantity: item.quantity } }
      );
    }
    await OrderModel.deleteOne({ order_id });
```

```
      res.json({
        success: true,
        message: "Order cancelled and stock updated. Payments made via Net Banking, UPI, or Card will be refunded within 2-3 business days."
       });


  } catch (err) {
    console.error("Cancel order error:", err);
    res.status(500).json({ success: false, message: "Server error while cancelling order." });
  }
});

app.post("/reset-password", async (req, res) => {
  const { email, newPassword } = req.body;

  try {
    const updated = await User.updateOne(
      { email },
      { $set: { password: newPassword } }
    );

    if (updated.modifiedCount === 0) {
      return res.status(400).json({ error: "Email not found or password is the same." });
    }

    res.json({ message: "Password updated successfully" });
  } catch (err) {
```

```
      console.error("Reset Password Error:", err);
      res.status(500).json({ error: "Internal server error" });
    }
});


app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

ADMIN :

ADMIN CATEGORIES :

FRUITS AND VEGETABLES ADMIN PAGE : f&v.html

```
<!DOCTYPE html>
<html>
<head>
   <title>Fruits and Vegetables</title>
   <link rel="stylesheet" href="cat.css">
   <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
</head>
<body>
   <nav class="topnav">
      <div class="logo">QuickMart</div>
      <input type="text" id="loc" placeholder="Enter Pincode" maxlength="6">
      <a class="NavLogo" href="homepg.html">🏠Home</a>
      <input type="text" id="search-bar" placeholder="Search QuickMart"
onkeyup="redirectToSearch(event)">
      <div class="user">
```

```
        <a class="NavLogo" href="login.html">Login 👤 </a>
      </div>
      <div class="cart">
        <a class="NavLogo" href="cart.html">Cart 🛒 <span id="cart-count"></span></a>
      </div>
      <div class="nav3" id="menu-toggle">☰</div>
    </nav>

    <div class="categories-nav">
      <a href="f&vadmin.html" class="active">Fruits & Vegetables</a>
      <a href="home&kitchenadmin.html">Home & Kitchen</a>
      <a href="packagedfoodsadmin.html">Packaged Foods</a>
      <a href="personalcareadmin.html">Personal Care</a>
    </div>

    <div class="container">
      <div class="sidebar">
        <b>Add New Product:</b><br><br>
        <form id="addProductForm" enctype="multipart/form-data">
          <input type="text" id="productName" placeholder="Product Name" required><br><br>
          <textarea id="productDesc" placeholder="Description" required></textarea><br><br>
          <input type="number" id="productPrice" placeholder="Price in Rupees" min="0"
required><br><br>
          <input type="number" id="productCurrentPrice" placeholder="Current Price in Rupees" min="0"
required><br><br>
          <input type="number" id="productStock" placeholder="Stock in Kg" min="0" required><br><br>
          <input type="file" id="productImage" required><br><br>
          <select id="productCategory">
            <option value="Fruits">Fruits</option>
            <option value="Vegetables">Vegetables</option>
```

```
            </select><br><br>
            <button type="submit" class="atc">Add Product</button><br><br><br><br><br>
        </form>
        <b>Filter by:</b><br>
        All Products<input type="checkbox" id="all-products" class="category" checked><br>
        Fruits<input type="checkbox" class="category" value="Fruits" name="Fruits"><br>
        Vegetables<input type="checkbox" class="category" value="Vegetables"
name="Vegetables"><br><br>
    </div>

    <div class="products" id="productList">
    </div>
  </div>
  <script>
function fetchProducts() {
  let pageCategory = "";
  if (window.location.pathname.includes("f&vadmin.html")) pageCategory = "CAT001";
  else if (window.location.pathname.includes("home&kitchenadmin.html")) pageCategory = "CAT002";
  else if (window.location.pathname.includes("packagedfoodsadmin.html")) pageCategory = "CAT003";
  else if (window.location.pathname.includes("personalcareadmin.html")) pageCategory = "CAT004";

  let subcategoryFilter = [];

  if ($("#all-products").prop("checked")) {
    subcategoryFilter = [];
  } else {
    $(".category:checked").each(function () {
      let subcat = $(this).val().trim();
      if (subcat) subcategoryFilter.push(encodeURIComponent(subcat));
```

```
    });
  }

  let queryParams = [`category=${pageCategory}`];

  if (subcategoryFilter.length > 0) {
    queryParams.push(`subcategories=${subcategoryFilter.join(",")}`);
  }

  let queryString = queryParams.length > 0 ? "?" + queryParams.join("&") : "";

  console.log("Final Query String Sent to Server:", queryString);

  $.get(`http://localhost:7000/products${queryString}`, (products) => {
    $("#productList").empty();

    if (products.length === 0) {
      $("#productList").html("<p>No products found</p>");
      return;
    }

    products.forEach((product) => {
      let productHTML = `
        <div class="product-box" data-id="${product._id}" data-category="${product.subcat}" data-price="${product.price}">
          <img src="http://localhost:7000/${product.image_url}" class="prod">
          <input type="text" class="product-name" value="${product.name}" disabled>
          <p>Description: <textarea class="desc-input" disabled>${product.description || 'No description'}</textarea></p>
          <p>Price: ₹<input type="number" class="price-input" value="${product.price}" disabled></p>
```

```
            <p>Current Price: ₹<input type="number" class="current-price-input"
value="${product.current_price}" disabled></p>

            <p>Stock: <input type="number" class="stock-input" value="${product.stock_quantity}"
disabled></p>

            <button class="update">Update</button>

          </div>

        `;

        $("#productList").append(productHTML);

      });

    }).fail((err) => {

      console.error("Error fetching products:", err);

    });

}


$(document).ready(() => {

  $("#all-products").change(function () {

    if ($(this).prop("checked")) {

      $(".category").prop("checked", false);

    }

    fetchProducts();

  });


  $(".category").change(function () {

    if ($(".category:checked").length > 0) {

      $("#all-products").prop("checked", false);

    } else {

      $("#all-products").prop("checked", true);

    }

    fetchProducts();

  });
```

```
  fetchProducts();
});

$("#addProductForm").submit(function (event) {
  event.preventDefault();

  let confirmation = confirm("Are you sure you want to add this product?");
  if (!confirmation) return;

  let category_id = "";
  if (window.location.pathname.includes("f&vadmin.html")) category_id = "CAT001";
  else if (window.location.pathname.includes("home&kitchenadmin.html")) category_id = "CAT002";
  else if (window.location.pathname.includes("packagedfoodsadmin.html")) category_id = "CAT003";
  else if (window.location.pathname.includes("personalcareadmin.html")) category_id = "CAT004";

  let price = parseFloat($("#productPrice").val());
  let current_price = parseFloat($("#productCurrentPrice").val());
  let stock_quantity = parseInt($("#productStock").val());

  if (
    isNaN(price) || price < 0 ||
    isNaN(current_price) || current_price < 0 ||
    isNaN(stock_quantity) || stock_quantity < 0
  ) {
    alert("Enter valid non-negative numeric values for price, current price, and stock.");
    return;
  }
```

```
if (current_price > price) {
   alert("Current price cannot be greater than the original price.");
   return;
}

let formData = new FormData();
formData.append("name", $("#productName").val().trim());
formData.append("description", $("#productDesc").val().trim());
formData.append("price", price);
formData.append("current_price", current_price);
formData.append("stock_quantity", stock_quantity);
formData.append("subcat", $("#productCategory").val());
formData.append("image", $("#productImage")[0].files[0]);
formData.append("category_id", category_id);

$.ajax({
   url: "http://localhost:7000/products",
   type: "POST",
   data: formData,
   processData: false,
   contentType: false,
   success: function () {
      alert("Product successfully added!");
      $("#addProductForm")[0].reset();
      fetchProducts();
   },
   error: function (xhr) {
      console.error("Error:", xhr.responseText);
      alert("Error adding product: " + xhr.responseText);
```

```
    }
  });
});


$(document).on("click", ".update", function () {
  let productBox = $(this).closest(".product-box");

  let inputs = productBox.find(".product-name, .desc-input, .price-input, .current-price-input, .stock-input");
  let button = $(this);

  if (button.text() === "Update") {
    inputs.prop("disabled", false);
    button.text("Save");
  } else if (confirm("Are you sure you want to update this product?")) {
    let updatedProduct = {
      _id: productId,
      name: productBox.find(".product-name").val().trim(),
      description: productBox.find(".desc-input").val().trim(),
      price: parseFloat(productBox.find(".price-input").val()),
      current_price: parseFloat(productBox.find(".current-price-input").val()),
      stock_quantity: parseInt(productBox.find(".stock-input").val())
    };

    if (
      isNaN(updatedProduct.price) || updatedProduct.price < 0 ||
      isNaN(updatedProduct.current_price) || updatedProduct.current_price < 0 ||
      isNaN(updatedProduct.stock_quantity) || updatedProduct.stock_quantity < 0
    ) {
```

```
      alert("Please enter valid non-negative numeric values for price, current price, and stock.");
      return;
    }

    $.ajax({
      url: `http://localhost:7000/products/${productId}`,
      type: "PUT",
      contentType: "application/json",
      data: JSON.stringify(updatedProduct),
      success: function (response) {
        alert("Product updated successfully!");
        inputs.prop("disabled", true);
        button.text("Update");

        productBox.attr("data-id", updatedProduct._id);
      },
      error: function (xhr) {
        console.error("Update Error:", xhr.responseText);
        alert("Error updating product: " + xhr.responseText);
      }
    });
  }
  $("#search-bar").keypress(function (event) {
    if (event.key === "Enter") {
      let query = $(this).val().trim();
      if (query) {
        window.location.href = `searchbaradmin.html?q=${encodeURIComponent(query)}`;
      }
    }
```

```
  });
});
  </script>
</body>
</html>
```

HOME AND KITCHEN : homeandkitchen.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Home and Kitchen</title>
  <link rel="stylesheet" href="cat.css">
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
</head>
<body>
  <nav class="topnav">
    <div class="logo">QuickMart</div>
    <input type="text" id="loc" placeholder="Enter Pincode" maxlength="6">
    <a class="NavLogo" href="homepg.html">🏠 Home</a>
    <input type="text" id="search-bar" placeholder="Search QuickMart"
onkeyup="redirectToSearch(event)">
    <div class="user">
      <a class="NavLogo" href="login.html">Login 👤 </a>
    </div>
    <div class="cart">
      <a class="NavLogo" href="cart.html">Cart 🛒 <span id="cart-count"></span></a>
    </div>
    <div class="nav3" id="menu-toggle">☰</div>
  </nav>
```

```
  <div class="categories-nav">
    <a href="f&vadmin.html">Fruits & Vegetables</a>
    <a href="home&kitchenadmin.html" class="active">Home & Kitchen</a>
    <a href="packagedfoodsadmin.html">Packaged Foods</a>
    <a href="personalcareadmin.html">Personal Care</a>
  </div>

  <div class="container">
    <div class="sidebar">
      <b>Add New Product:</b><br><br>
      <form id="addProductForm" enctype="multipart/form-data">
        <input type="text" id="productName" placeholder="Product Name" required><br><br>
        <textarea id="productDesc" placeholder="Description" required></textarea><br><br>
        <input type="number" id="productPrice" placeholder="Price in Rupees" min="0"
required><br><br>
        <input type="number" id="productCurrentPrice" placeholder="Current Price in Rupees" min="0"
required><br><br>
        <input type="number" id="productStock" placeholder="Stock in Kg" min="0" required><br><br>
        <input type="file" id="productImage" required><br><br>
        <select id="productCategory">
          <option value="Dal & Pulses">Dal & Pulses</option>
          <option value="Rice & Grains"> Rice & Grains</option>
          <option value="Cookware">Cookware</option>
          <option value=" Storage"> Storage</option>
          <option value=" Cleaning Supplies">Cleaning Supplies</option>
        </select><br><br>
        <button type="submit" class="atc">Add Product</button><br><br><br><br><br>
      </form>
      <b>Filter by:</b><br>
      All Products <input type="checkbox" id="all-products" class="category" checked><br>
```

```
        Dal & Pulses <input type="checkbox" class="category" value="Dal & Pulses"
name="subcategor"><br>

        Rice & Grains <input type="checkbox" class="category" value="Rice & Grains"
name="subcategory"><br>

        Cookware <input type="checkbox" class="category" value="Cookware" name="subcategory"><br>

        Storage <input type="checkbox" class="category" value="Storage" name="subcategory"><br>

        Cleaning Supplies <input type="checkbox" class="category" value="Cleaning Supplies"
name="subcategory"><br><br></br>

      </div>


      <div class="products" id="productList">

      </div>

    </div>

    <script>

function fetchProducts() {

    let pageCategory = "";

    if (window.location.pathname.includes("f&vadmin.html")) pageCategory = "CAT001";

    else if (window.location.pathname.includes("home&kitchenadmin.html")) pageCategory = "CAT002";

    else if (window.location.pathname.includes("packagedfoodsadmin.html")) pageCategory = "CAT003";

    else if (window.location.pathname.includes("personalcareadmin.html")) pageCategory = "CAT004";


    let subcategoryFilter = [];


    if ($("#all-products").prop("checked")) {

      subcategoryFilter = [];

    } else {

      $(".category:checked").each(function () {

        let subcat = $(this).val().trim();

        if (subcat) subcategoryFilter.push(encodeURIComponent(subcat));

      });
```

```
  }

  let queryParams = [`category=${pageCategory}`];

  if (subcategoryFilter.length > 0) {
    queryParams.push(`subcategories=${subcategoryFilter.join(",")}`);
  }

  let queryString = queryParams.length > 0 ? "?" + queryParams.join("&") : "";

  console.log("Final Query String Sent to Server:", queryString);

  $.get(`http://localhost:7000/products${queryString}`, (products) => {
    $("#productList").empty();

    if (products.length === 0) {
      $("#productList").html("<p>No products found</p>");
      return;
    }

    products.forEach((product) => {
      let productHTML = `
        <div class="product-box" data-id="${product._id}" data-category="${product.subcat}" data-price="${product.price}">
          <img src="http://localhost:7000/${product.image_url}" class="prod">
          <input type="text" class="product-name" value="${product.name}" disabled>
          <p>Description: <textarea class="desc-input" disabled>${product.description || 'No description'}</textarea></p>
          <p>Price: ₹<input type="number" class="price-input" value="${product.price}" disabled></p>
```

```
            <p>Current Price: ₹<input type="number" class="current-price-input"
value="${product.current_price}" disabled></p>

            <p>Stock: <input type="number" class="stock-input" value="${product.stock_quantity}"
disabled></p>

            <button class="update">Update</button>

          </div>

        `;

        $("#productList").append(productHTML);

      });

    }).fail((err) => {

      console.error("Error fetching products:", err);

    });

}


$(document).ready(() => {

  $("#all-products").change(function () {

    if ($(this).prop("checked")) {

      $(".category").prop("checked", false);

    }

    fetchProducts();

  });


  $(".category").change(function () {

    if ($(".category:checked").length > 0) {

      $("#all-products").prop("checked", false);

    } else {

      $("#all-products").prop("checked", true);

    }

    fetchProducts();

  });
```

```
  fetchProducts();
});

$("#addProductForm").submit(function (event) {
  event.preventDefault();

  let confirmation = confirm("Are you sure you want to add this product?");
  if (!confirmation) return;

  let category_id = "";
  if (window.location.pathname.includes("f&vadmin.html")) category_id = "CAT001";
  else if (window.location.pathname.includes("home&kitchenadmin.html")) category_id = "CAT002";
  else if (window.location.pathname.includes("packagedfoodsadmin.html")) category_id = "CAT003";
  else if (window.location.pathname.includes("personalcareadmin.html")) category_id = "CAT004";

  let price = parseFloat($("#productPrice").val());
  let current_price = parseFloat($("#productCurrentPrice").val());
  let stock_quantity = parseInt($("#productStock").val());

  if (
    isNaN(price) || price < 0 ||
    isNaN(current_price) || current_price < 0 ||
    isNaN(stock_quantity) || stock_quantity < 0
  ) {
    alert("Enter valid non-negative numeric values for price, current price, and stock.");
    return;
  }
```

```
if (current_price > price) {
   alert("Current price cannot be greater than the original price.");
   return;
}

let formData = new FormData();
formData.append("name", $("#productName").val().trim());
formData.append("description", $("#productDesc").val().trim());
formData.append("price", price);
formData.append("current_price", current_price);
formData.append("stock_quantity", stock_quantity);
formData.append("subcat", $("#productCategory").val());
formData.append("image", $("#productImage")[0].files[0]);
formData.append("category_id", category_id);

$.ajax({
   url: "http://localhost:7000/products",
   type: "POST",
   data: formData,
   processData: false,
   contentType: false,
   success: function () {
      alert("Product successfully added!");
      $("#addProductForm")[0].reset();
      fetchProducts();
   },
   error: function (xhr) {
      console.error("Error:", xhr.responseText);
      alert("Error adding product: " + xhr.responseText);
```

```
    }
  });
});


$(document).on("click", ".update", function () {
  let productBox = $(this).closest(".product-box");

  let productId = productBox.attr("data-id");
  let inputs = productBox.find(".product-name, .desc-input, .price-input, .current-price-input, .stock-input");
  let button = $(this);

  if (button.text() === "Update") {
    inputs.prop("disabled", false);
    button.text("Save");
  } else if (confirm("Are you sure you want to update this product?")) {
    let updatedProduct = {
      _id: productId,
      name: productBox.find(".product-name").val().trim(),
      description: productBox.find(".desc-input").val().trim(),
      price: parseFloat(productBox.find(".price-input").val()),
      current_price: parseFloat(productBox.find(".current-price-input").val()),
      stock_quantity: parseInt(productBox.find(".stock-input").val())
    };

    if (
      isNaN(updatedProduct.price) || updatedProduct.price < 0 ||
      isNaN(updatedProduct.current_price) || updatedProduct.current_price < 0 ||
      isNaN(updatedProduct.stock_quantity) || updatedProduct.stock_quantity < 0
```

```
    ) {
      alert("Please enter valid non-negative numeric values for price, current price, and stock.");
      return;
    }

    $.ajax({
      url: `http://localhost:7000/products/${productId}`,
      type: "PUT",
      contentType: "application/json",
      data: JSON.stringify(updatedProduct),
      success: function (response) {
        alert("Product updated successfully!");
        inputs.prop("disabled", true);
        button.text("Update");

        productBox.attr("data-id", updatedProduct._id);
      },
      error: function (xhr) {
        console.error("Update Error:", xhr.responseText);
        alert("Error updating product: " + xhr.responseText);
      }
    });
  }
});
  </script>
</body>
</html>
```

PACKAGE FOODS ADMIN : packagedfoodsadmin.html

```
<!DOCTYPE html>
```

```
<html>
<head>
  <title>Packages foods</title>
  <link rel="stylesheet" href="cat.css">
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
</head>
<body>
  <nav class="topnav">
    <div class="logo">QuickMart</div>
    <input type="text" id="category-search" placeholder="Search QuickMart">
    <div class="user">
      <a class="NavLogo" href="login.html">Login 👤 </a>
    </div>
    <div class="user">
      <a class="NavLogo" href="orderadmin.html">Orders</a>
    </div>
    </div>
  </nav>

  <div class="categories-nav">
    <a href="f&vadmin.html">Fruits & Vegetables</a>
    <a href="home&kitchenadmin.html">Home & Kitchen</a>
    <a href="packagedfoodsadmin.html" class="active">Packaged Foods</a>
    <a href="personalcareadmin.html">Personal Care</a>
  </div>

  <div class="container">
    <div class="sidebar">
      <b>Add New Product:</b><br><br>
```

```
        <form id="addProductForm" enctype="multipart/form-data">
          <input type="text" id="productName" placeholder="Product Name" required><br><br>
          <textarea id="productDesc" placeholder="Description" required></textarea><br><br>
          <input type="number" id="productPrice" placeholder="Price in Rupees" min="0"
required><br><br>
          <input type="number" id="productCurrentPrice" placeholder="Current Price in Rupees" min="0"
required><br><br>
          <input type="number" id="productStock" placeholder="Stock in Kg" min="0" required><br><br>
          <input type="file" id="productImage" required><br><br>
          <select id="productCategory">
            <option value="Frozen foods">Frozen foods</option>
            <option value="Biscuits and cookies">Biscuits and cookies</option>
            <option value="Drinks and Juices">Drinks and Juices</option>
            <option value="Breakfast Cereals">Breakfast Cereals</option>
            <option value="Ready foods">Ready foods</option>
          </select><br><br>
          <button type="submit" class="atc">Add Product</button><br><br><br><br><br>
        </form>
        <b>Filter by:</b><br>
          All Products<input type="checkbox" id="all-products" class="category" checked><br>
          Frozen foods<input type="checkbox" class="category" value="Frozen foods" name="Frozen
foods"><br>
          Biscuits and cookies<input type="checkbox" class="category" value="Biscuits and Cookies"
name="Biscuits and Cookies"><br>
          Drinks and Juices<input type="checkbox" class="category" value="Drinks and Juices"
name="Drinks and Juices"><br>
          Breakfast Cereals<input type="checkbox" class="category" value="Breakfast Cereals"
name="Breakfast Cereals"><br>
          Ready foods<input type="checkbox" class="category" value="Ready foods" name="Ready
foods"><br><br>
      </div>
```

```
    <div class="products" id="productList">
    </div>
  </div>
  <script>
function fetchProducts() {
  let pageCategory = "";
  if (window.location.pathname.includes("f&vadmin.html")) pageCategory = "CAT001";
  else if (window.location.pathname.includes("home&kitchenadmin.html")) pageCategory = "CAT002";
  else if (window.location.pathname.includes("packagedfoodsadmin.html")) pageCategory = "CAT003";
  else if (window.location.pathname.includes("personalcareadmin.html")) pageCategory = "CAT004";

  let subcategoryFilter = [];

  if ($("#all-products").prop("checked")) {
    subcategoryFilter = [];
  } else {
    $(".category:checked").each(function () {
      let subcat = $(this).val().trim();
      if (subcat) subcategoryFilter.push(encodeURIComponent(subcat));
    });
  }

  let queryParams = [`category=${pageCategory}`];

  if (subcategoryFilter.length > 0) {
    queryParams.push(`subcategories=${subcategoryFilter.join(",")}`);
  }

  let queryString = queryParams.length > 0 ? "?" + queryParams.join("&") : "";
```

```
  console.log("Final Query String Sent to Server:", queryString);


  $.get(` http://localhost:7000/products${queryString}` , (products) => {
    $("#productList").empty();


    if (products.length === 0) {
      $("#productList").html("<p>No products found</p>");
      return;
    }


    products.forEach((product) => {
      let productHTML = `
        <div class="product-box" data-id="${product._id}" data-category="${product.subcat}" data-price="${product.price}">
          <img src="http://localhost:7000/${product.image_url}" class="prod">
          <input type="text" class="product-name" value="${product.name}" disabled>
          <p>Description: <textarea class="desc-input" disabled>${product.description || 'No description'}</textarea></p>
          <p>Price: ₹<input type="number" class="price-input" value="${product.price}" disabled></p>
          <p>Current Price: ₹<input type="number" class="current-price-input" value="${product.current_price}" disabled></p>
          <p>Stock: <input type="number" class="stock-input" value="${product.stock_quantity}" disabled></p>
          <button class="update">Update</button>
        </div>
      `;
      $("#productList").append(productHTML);
    });
  }).fail((err) => {
    console.error("Error fetching products:", err);
```

```
  });
}

$(document).ready(() => {
  $("#all-products").change(function () {
    if ($(this).prop("checked")) {
      $(".category").prop("checked", false);
    }
    fetchProducts();
  });

  $(".category").change(function () {
    if ($(".category:checked").length > 0) {
      $("#all-products").prop("checked", false);
    } else {
      $("#all-products").prop("checked", true);
    }
    fetchProducts();
  });

  fetchProducts();
});
$("#category-search").on("keyup", function () {
    const searchVal = $(this).val().toLowerCase();

    $(".product-box").each(function () {
      const name = $(this).find(".product-name").val().toLowerCase();
      if (name.includes(searchVal)) {
        $(this).show();
```

```
        } else {
            $(this).hide();
        }
    });
});
$("#addProductForm").submit(function (event) {
    event.preventDefault();

    let confirmation = confirm("Are you sure you want to add this product?");
    if (!confirmation) return;

    let category_id = "";
    if (window.location.pathname.includes("f&vadmin.html")) category_id = "CAT001";
    else if (window.location.pathname.includes("home&kitchenadmin.html")) category_id = "CAT002";
    else if (window.location.pathname.includes("packagedfoodsadmin.html")) category_id = "CAT003";
    else if (window.location.pathname.includes("personalcareadmin.html")) category_id = "CAT004";

    let price = parseFloat($("#productPrice").val());
    let current_price = parseFloat($("#productCurrentPrice").val());
    let stock_quantity = parseInt($("#productStock").val());

    if (
        isNaN(price) || price < 0 ||
        isNaN(current_price) || current_price < 0 ||
        isNaN(stock_quantity) || stock_quantity < 0
    ) {
        alert("Enter valid non-negative numeric values for price, current price, and stock.");
        return;
    }
```

```
if (current_price > price) {
    alert("Current price cannot be greater than the original price.");
    return;
}

let formData = new FormData();
formData.append("name", $("#productName").val().trim());
formData.append("description", $("#productDesc").val().trim());
formData.append("price", price);
formData.append("current_price", current_price);
formData.append("stock_quantity", stock_quantity);
formData.append("subcat", $("#productCategory").val());
formData.append("image", $("#productImage")[0].files[0]);
formData.append("category_id", category_id);

$.ajax({
    url: "http://localhost:7000/products",
    type: "POST",
    data: formData,
    processData: false,
    contentType: false,
    success: function () {
        alert("Product successfully added!");
        $("#addProductForm")[0].reset();
        fetchProducts();
    },
    error: function (xhr) {
        console.error("Error:", xhr.responseText);
```

```
            alert("Error adding product: " + xhr.responseText);
        }
    });
});


$(document).on("click", ".update", function () {
    let productBox = $(this).closest(".product-box");

    let productId = productBox.attr("data-id");
    let inputs = productBox.find(".product-name, .desc-input, .price-input, .current-price-input, .stock-input");
    let button = $(this);

    if (button.text() === "Update") {
        inputs.prop("disabled", false);
        button.text("Save");
    } else if (confirm("Are you sure you want to update this product?")) {
        let updatedProduct = {
            _id: productId,
            name: productBox.find(".product-name").val().trim(),
            description: productBox.find(".desc-input").val().trim(),
            price: parseFloat(productBox.find(".price-input").val()),
            current_price: parseFloat(productBox.find(".current-price-input").val()),
            stock_quantity: parseInt(productBox.find(".stock-input").val())
        };

        if (
            isNaN(updatedProduct.price) || updatedProduct.price < 0 ||
            isNaN(updatedProduct.current_price) || updatedProduct.current_price < 0 ||
```

```
        isNaN(updatedProduct.stock_quantity) || updatedProduct.stock_quantity < 0
      ) {
        alert("Please enter valid non-negative numeric values for price, current price, and stock.");
        return;
      }

      $.ajax({
        url: `http://localhost:7000/products/${productId}`,
        type: "PUT",
        contentType: "application/json",
        data: JSON.stringify(updatedProduct),
        success: function (response) {
          alert("Product updated successfully!");
          inputs.prop("disabled", true);
          button.text("Update");

          productBox.attr("data-id", updatedProduct._id);
        },
        error: function (xhr) {
          console.error("Update Error:", xhr.responseText);
          alert("Error updating product: " + xhr.responseText);
        }
      });
    }
});
  </script>
</body>
</html>
```

PERSONAL CARE ADMIN PAGE : personalcareadmin.html

```
<!DOCTYPE html>

<html>

<head>

  <title>Personal Care</title>

  <link rel="stylesheet" href="cat.css">

  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>

</head>


<body>

  <nav class="topnav">

    <div class="logo">QuickMart</div>

    <input type="text" id="loc" placeholder="Enter Pincode" maxlength="6">

    <a class="NavLogo" href="homepg.html">🏠 Home</a>

    <input type="text" id="category-search" placeholder="Search QuickMart">

    <div class="user">

      <a class="NavLogo" href="login.html">Login 👤 </a>

    </div>

    <div class="cart">

      <a class="NavLogo" href="cart.html">Cart🛒 <span id="cart-count"></span></a>

    </div>

    <div class="nav3" id="menu-toggle">☰</div>

  </nav>


  <div class="categories-nav">

    <a href="f&vadmin.html">Fruits & Vegetables</a>

    <a href="home&kitchenadmin.html">Home & Kitchen</a>

    <a href="packagedfoodsadmin.html">Packaged Foods</a>

    <a href="personalcareadmin.html" class="active">Personal Care</a>
```

```
  </div>


  <div class="container">
    <div class="sidebar">
      <b>Add New Product:</b><br><br>
      <form id="addProductForm" enctype="multipart/form-data">
        <input type="text" id="productName" placeholder="Product Name" required><br><br>
        <textarea id="productDesc" placeholder="Description" required></textarea><br><br>
        <input type="number" id="productPrice" placeholder="Price in Rupees" min="0"
required><br><br>
        <input type="number" id="productCurrentPrice" placeholder="Current Price in Rupees" min="0"
required><br><br>
        <input type="number" id="productStock" placeholder="Stock in Kg" min="0" required><br><br>
        <input type="file" id="productImage" required><br><br>
        <select id="productCategory">
          <option value="Body Care">Body Care</option>
          <option value="Face Wash">Face Wash</option>
          <option value="Shampoo">Shampoo</option>
          <option value="Conditioner">Conditioner</option>
          <option value="Oral Care">Oral Care</option>
        </select><br><br>
        <button type="submit" class="atc">Add Product</button><br><br><br><br><br>
      </form>
      <b>Filter by:</b><br>
        All Products<input type="checkbox" id="all-products" class="category" checked><br>
        Body Care<input type="checkbox" class="category" value="Body Care" name="Body Care"><br>
        Face Wash<input type="checkbox" class="category" value="Face Wash" name="Face
Wash"><br>
        Shampoo<input type="checkbox" class="category" value="Shampoo" name="Shampoo"><br>
        Conditioner<input type="checkbox" class="category" value="Conditioner"
name="Conditioner"><br>
```

```
        Oral Care<input type="checkbox" class="category" value="Oral Care" name="Oral Care"><br><br>

    </div>

    <div class="products" id="productList">

    </div>

  </div>

  <script>

    function fetchProducts() {

      let pageCategory = "";

      if (window.location.pathname.includes("f&vadmin.html")) pageCategory = "CAT001";

      else if (window.location.pathname.includes("home&kitchenadmin.html")) pageCategory = "CAT002";

      else if (window.location.pathname.includes("packagedfoodsadmin.html")) pageCategory = "CAT003";

      else if (window.location.pathname.includes("personalcareadmin.html")) pageCategory = "CAT004";


      let subcategoryFilter = [];


      if ($("#all-products").prop("checked")) {

        subcategoryFilter = [];

      } else {

        $(".category:checked").each(function () {

          let subcat = $(this).val().trim();

          if (subcat) subcategoryFilter.push(encodeURIComponent(subcat));

        });

      }


      let queryParams = [`category=${pageCategory}`];
```

```
    if (subcategoryFilter.length > 0) {
      queryParams.push(`subcategories=${subcategoryFilter.join(",")}`);
    }

    let queryString = queryParams.length > 0 ? "?" + queryParams.join("&") : "";

    console.log("Final Query String Sent to Server:", queryString);

    $.get(`http://localhost:7000/products${queryString}`, (products) => {
      $("#productList").empty();

      if (products.length === 0) {
        $("#productList").html("<p>No products found</p>");
        return;
      }

      products.forEach((product) => {
        let productHTML = `
          <div class="product-box" data-id="${product._id}" data-category="${product.subcat}" data-price="${product.price}">
            <img src="http://localhost:7000/${product.image_url}" class="prod">
            <input type="text" class="product-name" value="${product.name}" disabled>
            <p>Description: <textarea class="desc-input" disabled>${product.description || 'No description'}</textarea></p>
            <p>Price: ₹<input type="number" class="price-input" value="${product.price}" disabled></p>
            <p>Current Price: ₹<input type="number" class="current-price-input" value="${product.current_price}" disabled></p>
            <p>Stock: <input type="number" class="stock-input" value="${product.stock_quantity}" disabled></p>
            <button class="update">Update</button>
```

```
            </div>
          `;
          $("#productList").append(productHTML);
        });
    }).fail((err) => {
        console.error("Error fetching products:", err);
    });
}




$(document).ready(() => {
    $("#all-products").change(function () {
        if ($(this).prop("checked")) {
            $(".category").prop("checked", false);
        }
        fetchProducts();
    });

    $(".category").change(function () {
        if ($(".category:checked").length > 0) {
            $("#all-products").prop("checked", false);
        } else {
            $("#all-products").prop("checked", true);
        }
        fetchProducts();
    });

    fetchProducts();
```

```
});

$("#addProductForm").submit(function (event) {
  event.preventDefault();

  let confirmation = confirm("Are you sure you want to add this product?");
  if (!confirmation) return;

  let category_id = "";
  if (window.location.pathname.includes("f&vadmin.html")) category_id = "CAT001";
  else if (window.location.pathname.includes("home&kitchenadmin.html")) category_id = "CAT002";
  else if (window.location.pathname.includes("packagedfoodsadmin.html")) category_id =
"CAT003";
  else if (window.location.pathname.includes("personalcareadmin.html")) category_id = "CAT004";

  let price = parseFloat($("#productPrice").val());
  let current_price = parseFloat($("#productCurrentPrice").val());
  let stock_quantity = parseInt($("#productStock").val());

  if (
    isNaN(price) || price < 0 ||
    isNaN(current_price) || current_price < 0 ||
    isNaN(stock_quantity) || stock_quantity < 0
  ) {
    alert("Enter valid non-negative numeric values for price, current price, and stock.");
    return;
  }

  if (current_price > price) {
```

```
    alert("Current price cannot be greater than the original price.");
    return;
}

let formData = new FormData();
formData.append("name", $("#productName").val().trim());
formData.append("description", $("#productDesc").val().trim());
formData.append("price", price);
formData.append("current_price", current_price);
formData.append("stock_quantity", stock_quantity);
formData.append("subcat", $("#productCategory").val());
formData.append("image", $("#productImage")[0].files[0]);
formData.append("category_id", category_id);

$.ajax({
    url: "http://localhost:7000/products",
    type: "POST",
    data: formData,
    processData: false,
    contentType: false,
    success: function () {
        alert("Product successfully added!");
        $("#addProductForm")[0].reset();
        fetchProducts();
    },
    error: function (xhr) {
        console.error("Error:", xhr.responseText);
        alert("Error adding product: " + xhr.responseText);
    }
```

```
    });
  });
  $("#category-search").on("keyup", function () {
  const searchVal = $(this).val().toLowerCase();
  let anyVisible = false;

  $(".product-box").each(function () {
    const name = $(this).find(".product-name").val().toLowerCase();
    if (name.includes(searchVal)) {
      $(this).show();
      anyVisible = true;
    } else {
      $(this).hide();
    }
  });

  if (!anyVisible) {
    if ($("#no-results").length === 0) {
      $("#productList").append("<p id='no-results'>No results found</p>");
    }
  } else {
    $("#no-results").remove();
  }
});
  $(document).on("click", ".update", function () {
    let productBox = $(this).closest(".product-box");

    let productId = productBox.attr("data-id");
```

```
        let inputs = productBox.find(".product-name, .desc-input, .price-input, .current-price-input, .stock-input");
        let button = $(this);

        if (button.text() === "Update") {
            inputs.prop("disabled", false);
            button.text("Save");
        } else if (confirm("Are you sure you want to update this product?")) {
            let updatedProduct = {
                _id: productId,
                name: productBox.find(".product-name").val().trim(),
                description: productBox.find(".desc-input").val().trim(),
                price: parseFloat(productBox.find(".price-input").val()),
                current_price: parseFloat(productBox.find(".current-price-input").val()),
                stock_quantity: parseInt(productBox.find(".stock-input").val())
            };

            if (
                isNaN(updatedProduct.price) || updatedProduct.price < 0 ||
                isNaN(updatedProduct.current_price) || updatedProduct.current_price < 0 ||
                isNaN(updatedProduct.stock_quantity) || updatedProduct.stock_quantity < 0
            ) {
                alert("Please enter valid non-negative numeric values for price, current price, and stock.");
                return;
            }

            $.ajax({
                url: `http://localhost:7000/products/${productId}`,
                type: "PUT",
```

```
            contentType: "application/json",
            data: JSON.stringify(updatedProduct),
            success: function (response) {
              alert("Product updated successfully!");
              inputs.prop("disabled", true);
              button.text("Update");

              productBox.attr("data-id", updatedProduct._id);
            },
            error: function (xhr) {
              console.error("Update Error:", xhr.responseText);
              alert("Error updating product: " + xhr.responseText);
            }
          });
        }
      });
        </script>
</body>
</html>
```

ADMIN BACKEND PAGES :

adminscript.js

```
function fetchProducts() {
  let pageCategory = "";
  if (window.location.pathname.includes("f&vadmin.html")) pageCategory = "CAT001";
  else if (window.location.pathname.includes("home&kitchenadmin.html")) pageCategory = "CAT002";
  else if (window.location.pathname.includes("packagedfoodsadmin.html")) pageCategory = "CAT003";
  else if (window.location.pathname.includes("personalcareadmin.html")) pageCategory = "CAT004";

  let subcategoryFilter = [];
```

```
if ($("#all-products").prop("checked")) {
   subcategoryFilter = [];
} else {
   $(".category:checked").each(function () {
      let subcat = $(this).val().trim();
      if (subcat) subcategoryFilter.push(encodeURIComponent(subcat));
   });
}

let queryParams = [`category=${pageCategory}`];

if (subcategoryFilter.length > 0) {
   queryParams.push(`subcategories=${subcategoryFilter.join(",")}`);
}

let queryString = queryParams.length > 0 ? "?" + queryParams.join("&") : "";

console.log("Final Query String Sent to Server:", queryString);

$.get(`http://localhost:7000/products${queryString}`, (products) => {
   $("#productList").empty();

   if (products.length === 0) {
      $("#productList").html("<p>No products found</p>");
      return;
   }

   products.forEach((product) => {
```

```
        let productHTML = `
          <div class="product-box" data-id="${product._id}" data-category="${product.subcat}" data-price="${product.price}">
            <img src="http://localhost:7000/${product.image_url}" class="prod">
            <input type="text" class="product-name" value="${product.name}" disabled>
            <p>Description: <textarea class="desc-input" disabled>${product.description || 'No description'}</textarea></p>
            <p>Price: ₹<input type="number" class="price-input" value="${product.price}" disabled></p>
            <p>Current Price: ₹<input type="number" class="current-price-input" value="${product.current_price}" disabled></p>
            <p>Stock: <input type="number" class="stock-input" value="${product.stock_quantity}" disabled></p>
            <button class="update">Update</button>
          </div>
        `;
        $("#productList").append(productHTML);
      });
    }).fail((err) => {
      console.error("Error fetching products:", err);
    });
}

$(document).ready(() => {
  $("#all-products").change(function () {
    if ($(this).prop("checked")) {
      $(".category").prop("checked", false);
    }
    fetchProducts();
  });
```

```
    $(".category").change(function () {
        if ($(".category:checked").length > 0) {
            $("#all-products").prop("checked", false);
        } else {
            $("#all-products").prop("checked", true);
        }
        fetchProducts();
    });

    fetchProducts();
});

$("#addProductForm").submit(function (event) {
    event.preventDefault();

    let confirmation = confirm("Are you sure you want to add this product?");
    if (!confirmation) return;

    let category_id = "";
    if (window.location.pathname.includes("f&vadmin.html")) category_id = "CAT001";
    else if (window.location.pathname.includes("home&kitchenadmin.html")) category_id = "CAT002";
    else if (window.location.pathname.includes("packagedfoodsadmin.html")) category_id = "CAT003";
    else if (window.location.pathname.includes("personalcareadmin.html")) category_id = "CAT004";

    let price = parseFloat($("#productPrice").val());
    let current_price = parseFloat($("#productCurrentPrice").val());
    let stock_quantity = parseInt($("#productStock").val());

    if (
```

```
    isNaN(price) || price < 0 ||
    isNaN(current_price) || current_price < 0 ||
    isNaN(stock_quantity) || stock_quantity < 0
) {
    alert("Enter valid non-negative numeric values for price, current price, and stock.");
    return;
}

if (current_price > price) {
    alert("Current price cannot be greater than the original price.");
    return;
}

let formData = new FormData();
formData.append("name", $("#productName").val().trim());
formData.append("description", $("#productDesc").val().trim());
formData.append("price", price);
formData.append("current_price", current_price);
formData.append("stock_quantity", stock_quantity);
formData.append("subcat", $("#productCategory").val());
formData.append("image", $("#productImage")[0].files[0]);
formData.append("category_id", category_id);

$.ajax({
    url: "http://localhost:7000/products",
    type: "POST",
    data: formData,
    processData: false,
    contentType: false,
```

```
      success: function () {
        alert("Product successfully added!");
        $("#addProductForm")[0].reset();
        fetchProducts();
      },
      error: function (xhr) {
        console.error("Error:", xhr.responseText);
        alert("Error adding product: " + xhr.responseText);
      }
    });
});
$("#category-search").on("keyup", function () {
const searchVal = $(this).val().toLowerCase();
let anyVisible = false;

$(".product-box").each(function () {
  const name = $(this).find(".product-name").val().toLowerCase();
  if (name.includes(searchVal)) {
    $(this).show();
    anyVisible = true;
  } else {
    $(this).hide();
  }
});

if (!anyVisible) {
  if ($("#no-results").length === 0) {
    $("#productList").append("<p id='no-results'>No results found</p>");
  }
```

```
} else {
   $("#no-results").remove();
}
});
$(document).on("click", ".update", function () {
   let productBox = $(this).closest(".product-box");

   let productId = productBox.attr("data-id");
   let inputs = productBox.find(".product-name, .desc-input, .price-input, .current-price-input, .stock-input");
   let button = $(this);

   if (button.text() === "Update") {
      inputs.prop("disabled", false);
      button.text("Save");
   } else if (confirm("Are you sure you want to update this product?")) {
      let updatedProduct = {
         _id: productId,
         name: productBox.find(".product-name").val().trim(),
         description: productBox.find(".desc-input").val().trim(),
         price: parseFloat(productBox.find(".price-input").val()),
         current_price: parseFloat(productBox.find(".current-price-input").val()),
         stock_quantity: parseInt(productBox.find(".stock-input").val())
      };

      if (
         isNaN(updatedProduct.price) || updatedProduct.price < 0 ||
         isNaN(updatedProduct.current_price) || updatedProduct.current_price < 0 ||
         isNaN(updatedProduct.stock_quantity) || updatedProduct.stock_quantity < 0
```

```
    ) {
      alert("Please enter valid non-negative numeric values for price, current price, and stock.");
      return;
    }

    $.ajax({
      url: `http://localhost:7000/products/${productId}`,
      type: "PUT",
      contentType: "application/json",
      data: JSON.stringify(updatedProduct),
      success: function (response) {
        alert("Product updated successfully!");
        inputs.prop("disabled", true);
        button.text("Update");

        productBox.attr("data-id", updatedProduct._id);
      },
      error: function (xhr) {
        console.error("Update Error:", xhr.responseText);
        alert("Error updating product: " + xhr.responseText);
      }
    });
  }
});
```

Adminserver.js

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
```

```
const fileUpload = require("express-fileupload");
const path = require("path");
const PORT = 7000;

const app = express();
app.use(express.json());
app.use(cors());
app.use(fileUpload());
app.use("/", express.static(__dirname));

mongoose.connect("mongodb://127.0.0.1:27017/Groceries")
  .then(() => console.log("Connected to MongoDB"))
  .catch(err => console.error("MongoDB Connection Error:", err));

const productSchema = new mongoose.Schema({
    _id: String,
    name: String,
    description: String,
    price: Number,
    current_price: Number,
    stock_quantity: Number,
    category_id: String,
    subcat: String,
    image_url: String
});

const Product = mongoose.model('Product', productSchema,'Products');

app.post("/products", async (req, res) => {
```

```
try {
  console.log("Received Request:", req.body);

  let { name, description, price, current_price, stock_quantity, subcat, category_id } = req.body;
  let image_url = "default.jpg";

  if (!name || !description || isNaN(price) || isNaN(current_price) || isNaN(stock_quantity) || !subcat) {
    return res.status(400).json({ error: "Please fill all required fields correctly." });
  }

  if (!category_id) {
    return res.status(400).json({ error: "Category ID is required." });
  }

  const existingProduct = await Product.findOne({ name: { $regex: new RegExp(`^${name}$`, 'i') } });
  if (existingProduct) {
    return res.status(409).json({ error: "Product with this name already exists." });
  }

  const products = await Product.find();
  let highestProductId = 0;
  if (products.length > 0) {
    highestProductId = Math.max(...products.map(p => parseInt(p._id.substring(1))));
  }
  const nextId = `P${String(highestProductId + 1).padStart(3, "0")}`;

  if (req.files && req.files.image) {
    let imageFile = req.files.image;
    const fileName = imageFile.name;
```

```
    const fileExt = path.extname(fileName).toLowerCase();

    if (![".jpg", ".jpeg", ".png"].includes(fileExt)) {
      return res.status(400).json({ error: "Only .jpg, .jpeg, .png files allowed." });
    }

    const maxSize = 5 * 1024 * 1024;
    if (imageFile.size > maxSize) {
      return res.status(400).json({ error: "Image size must be 5MB or less." });
    }

    image_url = fileName;
    const uploadPath = path.join(__dirname,  image_url);

    try {
      await imageFile.mv(uploadPath);
      console.log("Image uploaded:", image_url);
    } catch (err) {
      console.error("Upload failed:", err);
      return res.status(500).json({ error: "Image upload failed." });
    }
  }

  const newProduct = new Product({
    _id: nextId,
    name,
    description,
    price,
    current_price,
```

```
      stock_quantity,
      subcat,
      category_id,
      image_url
    });

    const savedProduct = await newProduct.save();
    console.log("Product added:", savedProduct);
    res.status(201).json({ message: "Product added successfully!", product: savedProduct });

  } catch (error) {
    console.error("Product Insertion Error:", error);
    res.status(500).json({ error: "Server error", details: error.message });
  }
});

app.get("/products", async (req, res) => {
  try {
    const { category, subcategories } = req.query;

    let filter = {};

    if (category) {
      filter.category_id = category;
    }

    if (subcategories) {
      const subcatArray = subcategories.split(",").map(s => decodeURIComponent(s.trim()));
      filter.subcat = { $in: subcatArray };
```

```
    }

    const products = await Product.find(filter);
    res.json(products);
  } catch (error) {
    console.error("Error fetching products:", error);
    res.status(500).json({ error: "Server error while fetching products." });
  }
});

app.put("/products/:id", async (req, res) => {
  try {
    let productId = req.params.id;
    let updatedData = req.body;

    const result = await Product.updateOne(
      { _id: productId },
      { $set: updatedData }
    );

    if (result.modifiedCount > 0) {
      res.json({ message: "Product updated successfully" });
    } else {
      res.status(400).json({ message: "No changes made or product not found" });
    }
  } catch (error) {
    console.error("Update Error:", error);
    res.status(500).json({ error: error.message });
  }
```

```
});



app.listen(PORT, () => {
   console.log(`Server running at http://localhost:${PORT}` );
 });


atc.js

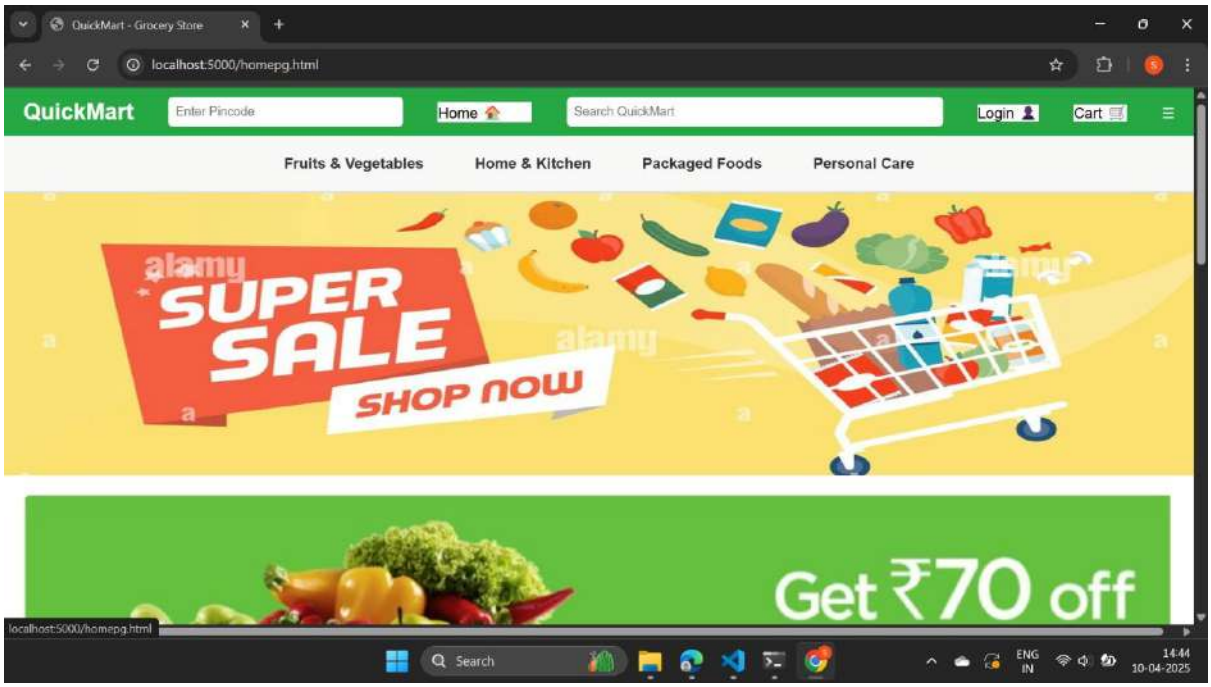
let cart={};
$(document).ready(function(){
   $(document).on("click",".atc",function(){
      alert("Added to cart!");
      let prodBox=$(this).closest(".product-box");
      let prodName=prodBox[0].querySelector("p").innerHTML.split("<br>")[0].trim();
      let prodPrice=prodBox.data("price");
      if(cart[prodName]){
         cart[prodName].qty+=1;
      }else{
         cart[prodName]={ price:prodPrice,qty:1};
      }


      updateCartCount();
   });
});
function updateCartCount(){
   let totalItems=0;
   for (let key in cart){
      totalItems+=cart[key].qty;
   }
```

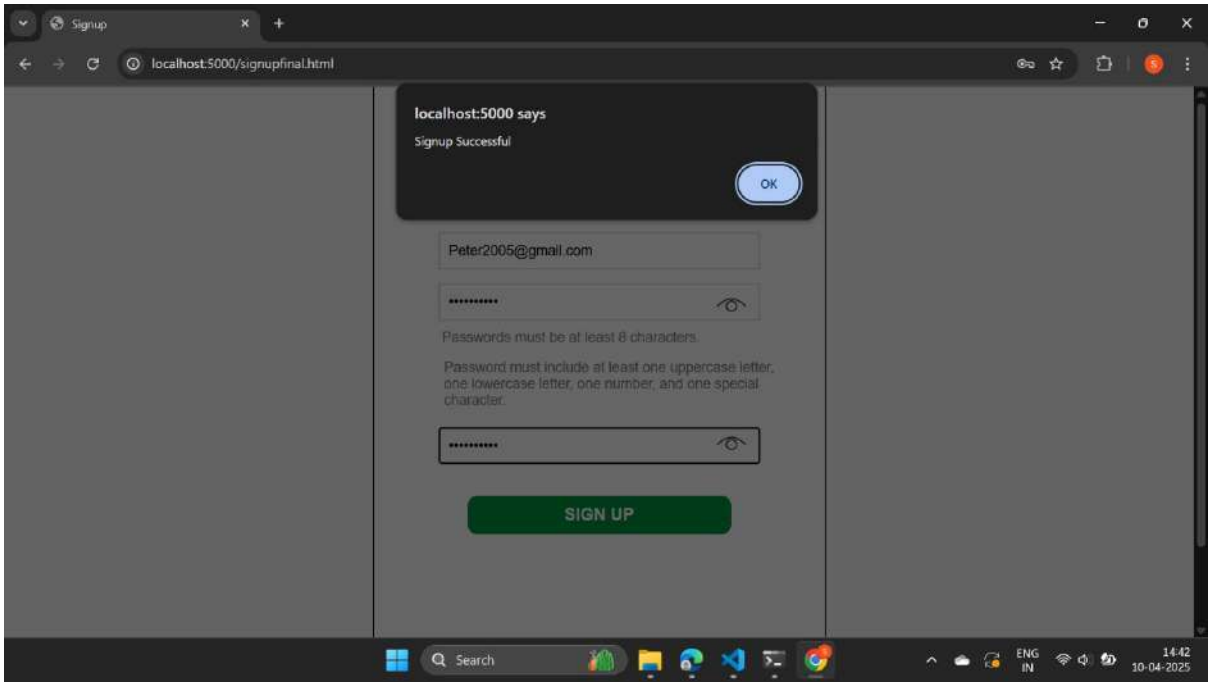
```
    $("#cart-count").text(totalItems);
}
```

SAMPLE SCREENSHOTS:

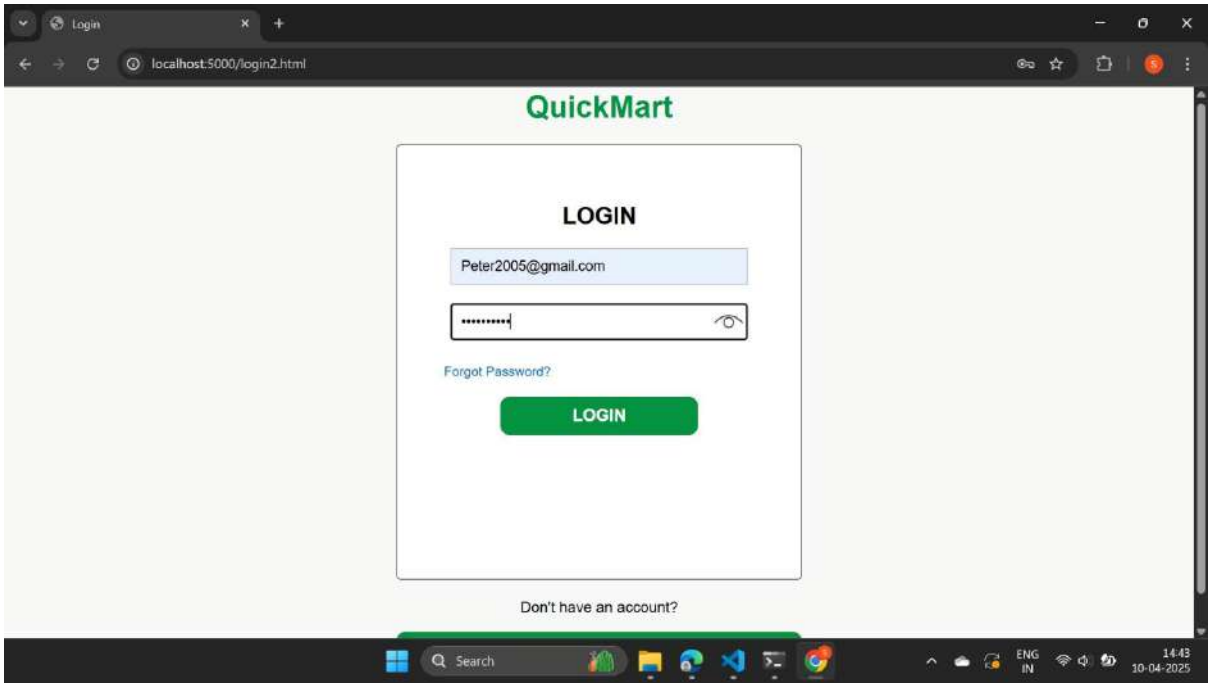
HOME PAGE :



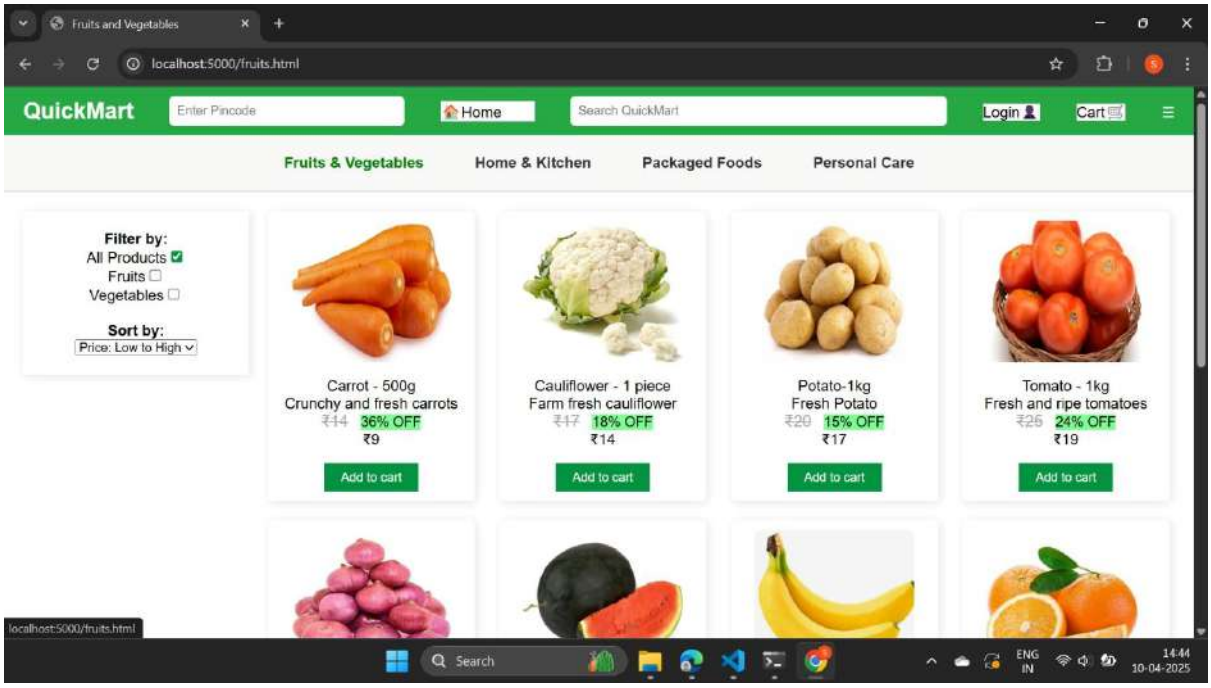
SIGNUP PAGE:

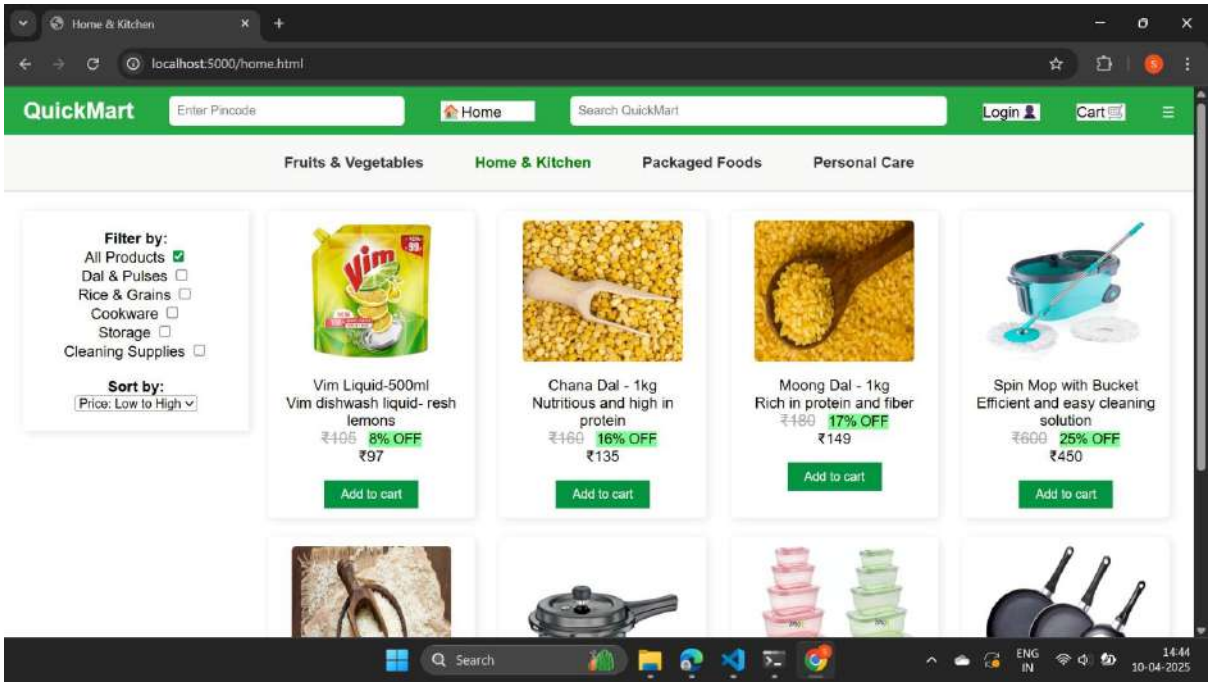


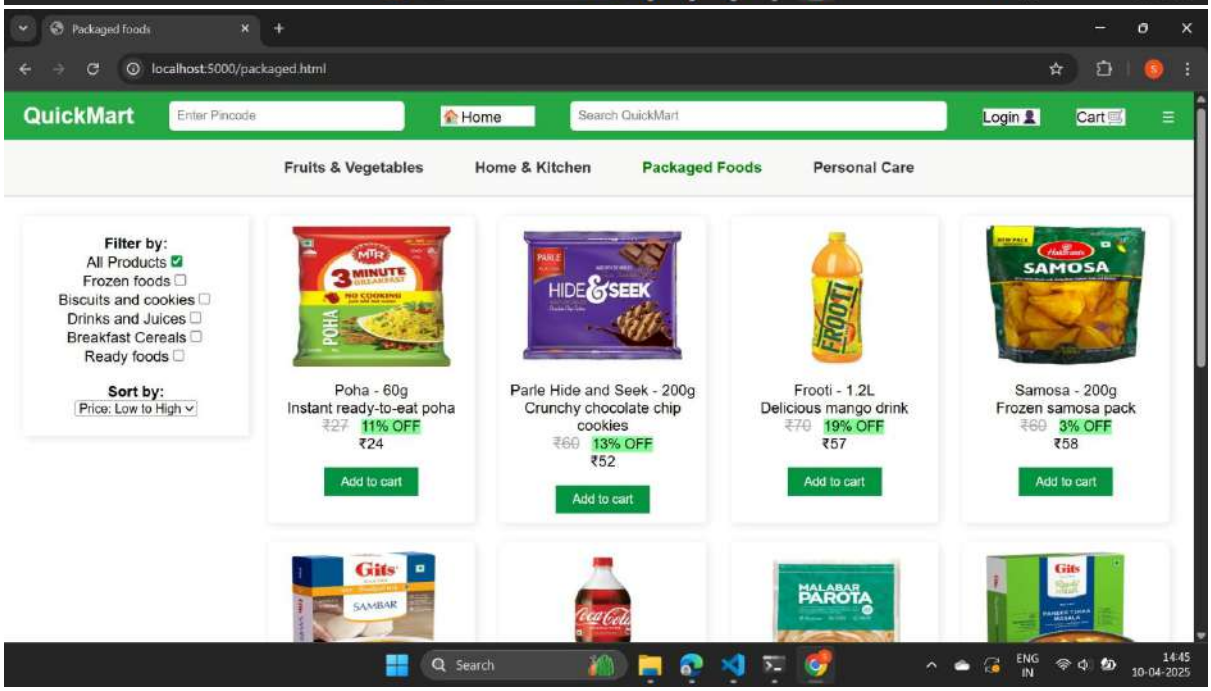
LOGIN :

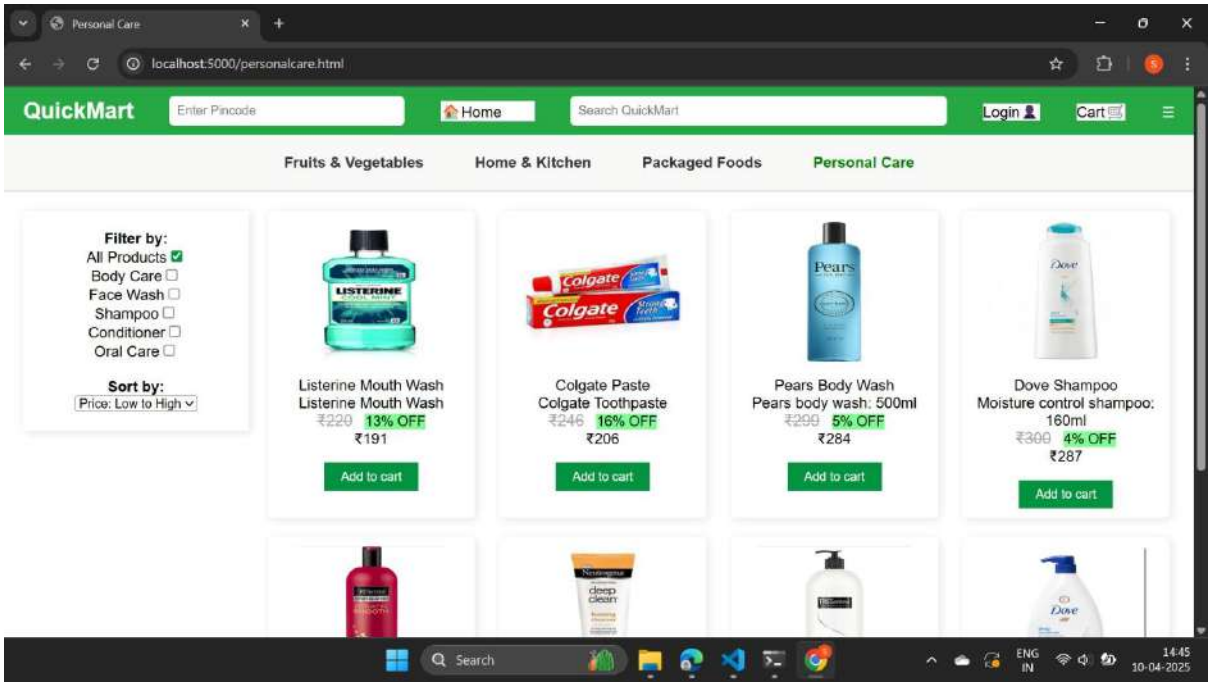


CATEGORIES :

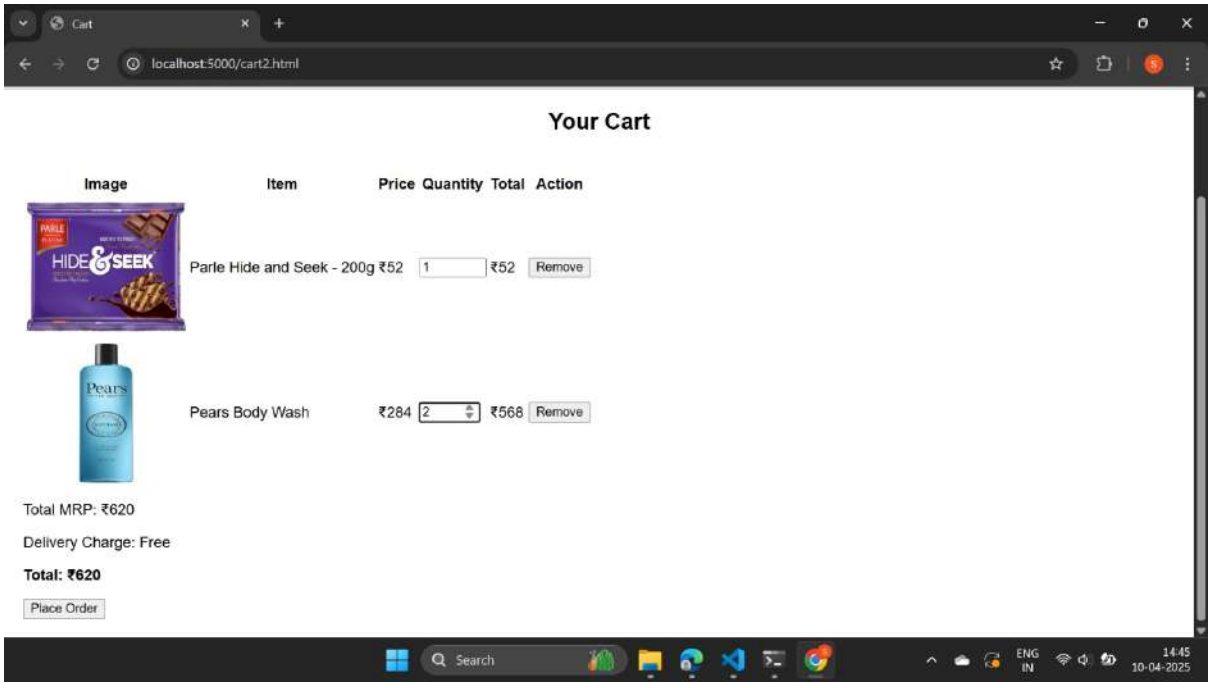



Home & Kitchen
localhost:5000/home.html
QuickMart
Enter Pincode
Home
Search QuickMart
Login
Cart
Fruits & Vegetables
Home & Kitchen
Packaged Foods
Personal Care
Filter by:
All Products
Dal & Pulses
Rice & Grains
Cookware
Storage
Cleaning Supplies
Sort by:
Price: Low to High
Vim Liquid-500ml
Vim dishwash liquid- resh lemons
₹105 8% OFF
₹97
Add to cart
Chana Dal - 1kg
Nutritious and high in protein
₹160 16% OFF
₹135
Add to cart
Moong Dal - 1kg
Rich in protein and fiber
₹180 17% OFF
₹149
Add to cart
Spin Mop with Bucket
Efficient and easy cleaning solution
₹600 25% OFF
₹450
Add to cart
Search
ENG
IN
14:44
10-04-2025

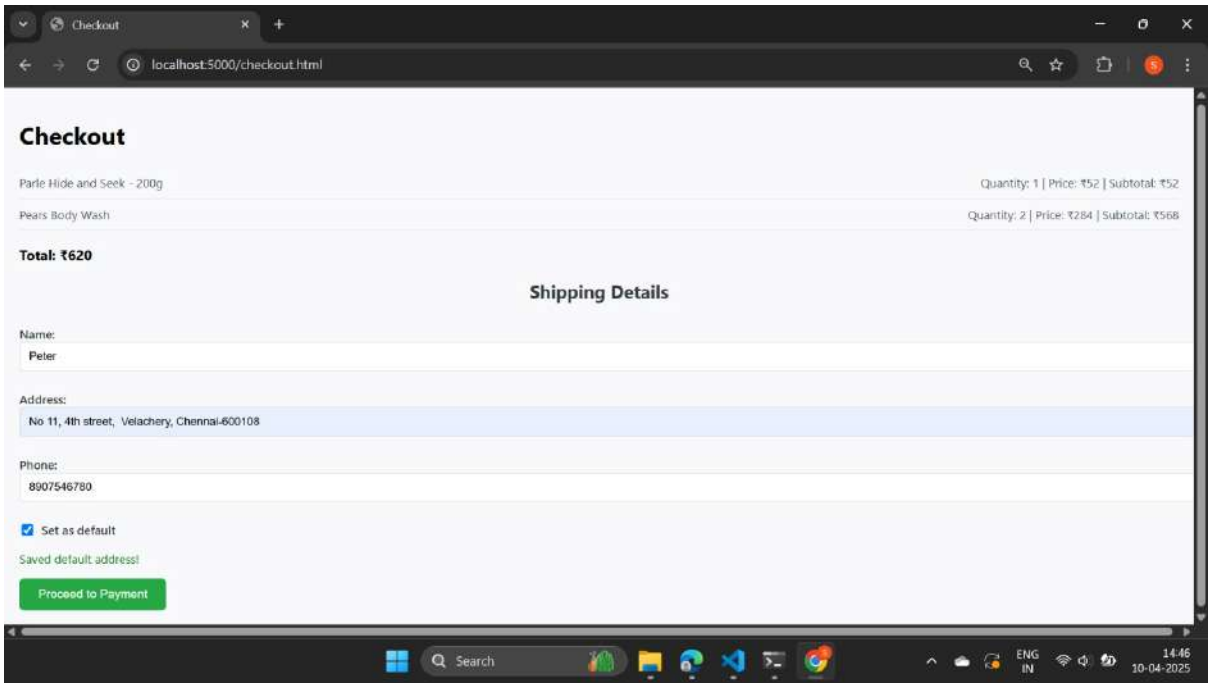

Packaged Foods
localhost:5000/packaged.html
QuickMart
Enter Pincode
Home
Search QuickMart
Login
Cart
Fruits & Vegetables
Home & Kitchen
Packaged Foods
Personal Care
Filter by:
All Products
Frozen foods
Biscuits and cookies
Drinks and Juices
Breakfast Cereals
Ready foods
Sort by:
Price: Low to High
Poha - 60g
Instant ready-to-eat poha
₹27 11% OFF
₹24
Add to cart
Parle Hide and Seek - 200g
Crunchy chocolate chip cookies
₹60 13% OFF
₹52
Add to cart
Frooti - 1.2L
Delicious mango drink
₹70 19% OFF
₹57
Add to cart
Samosa - 200g
Frozen samosa pack
₹60 3% OFF
₹58
Add to cart
Search
ENG
IN
14:45
10-04-2025


Personal Care
localhost:5000/personalcare.html
QuickMart
Enter Pincode
Home
Search QuickMart
Login
Cart
Fruits & Vegetables
Home & Kitchen
Packaged Foods
Personal Care
Filter by:
All Products
Body Care
Face Wash
Shampoo
Conditioner
Oral Care
Sort by:
Price: Low to High
Listerine Mouth Wash
Listerine Mouth Wash
₹220 13% OFF
₹191
Add to cart
Colgate Paste
Colgate Toothpaste
₹246 16% OFF
₹206
Add to cart
Pears Body Wash
Pears body wash: 500ml
₹299 5% OFF
₹284
Add to cart
Dove Shampoo
Moisture control shampoo: 160ml
₹300 4% OFF
₹287
Add to cart
Search
ENG
IN
14:45
10-04-2025

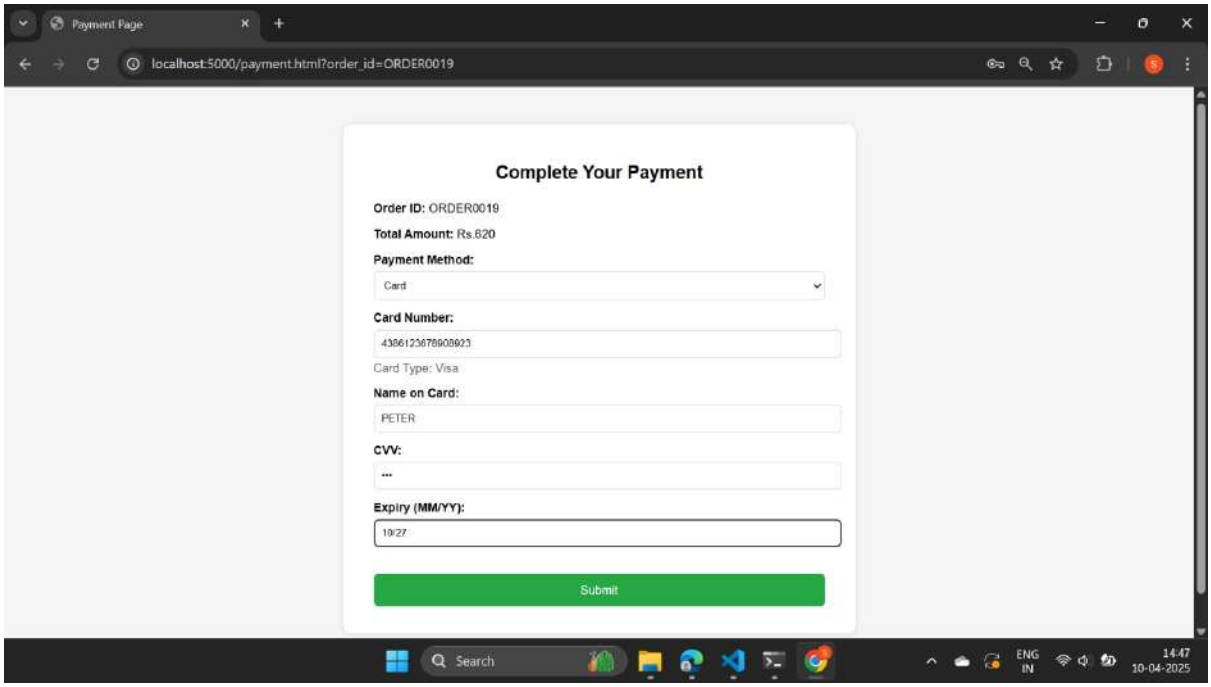
YOUR CART :


Cart
localhost:5000/cart2.html
Your Cart
Image
Item
Price
Quantity
Total
Action
Parle Hide and Seek - 200g ₹52
1
₹52
Remove
Pears Body Wash
₹284
2
₹568
Remove
Total MRP: ₹620
Delivery Charge: Free
Total: ₹620
Place Order
Search
ENG
IN
14:45
10-04-2025

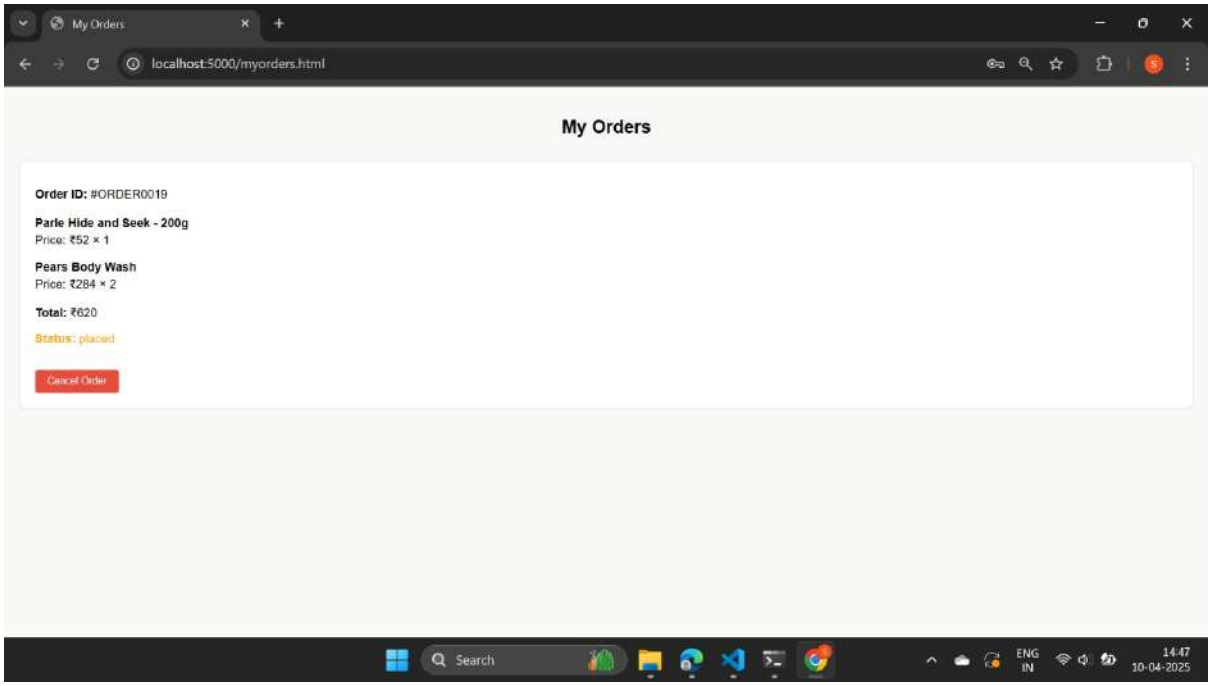
CHECKOUT PAGE :



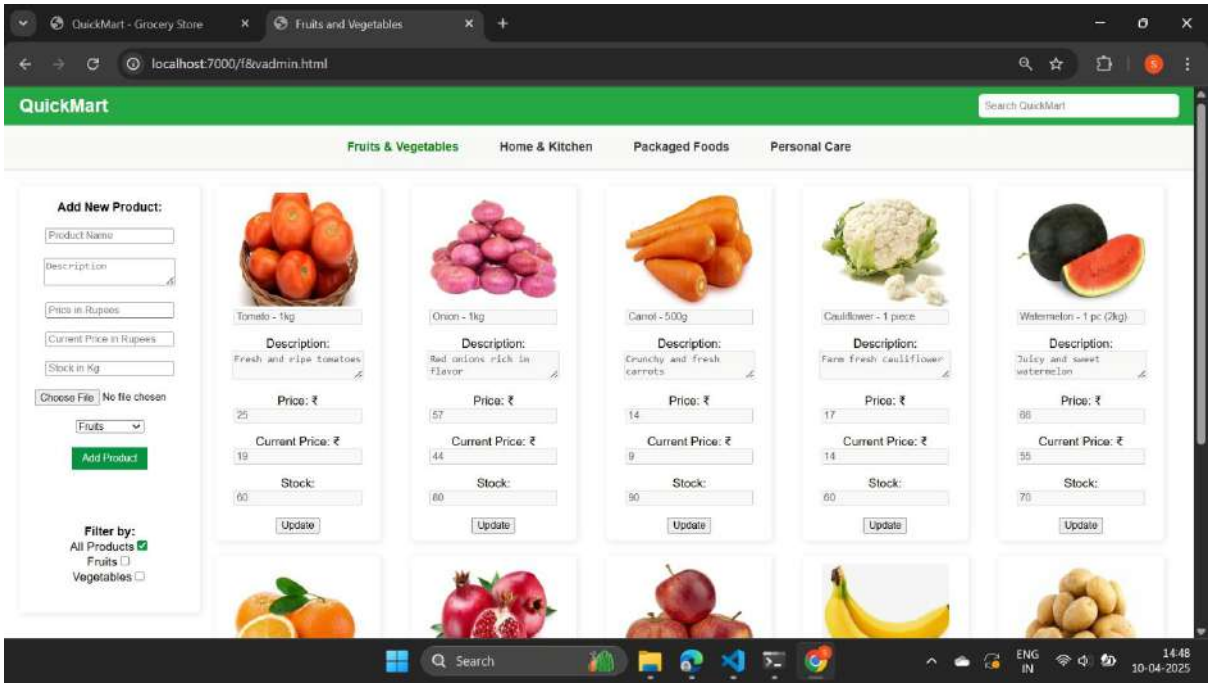
PAYMENT :

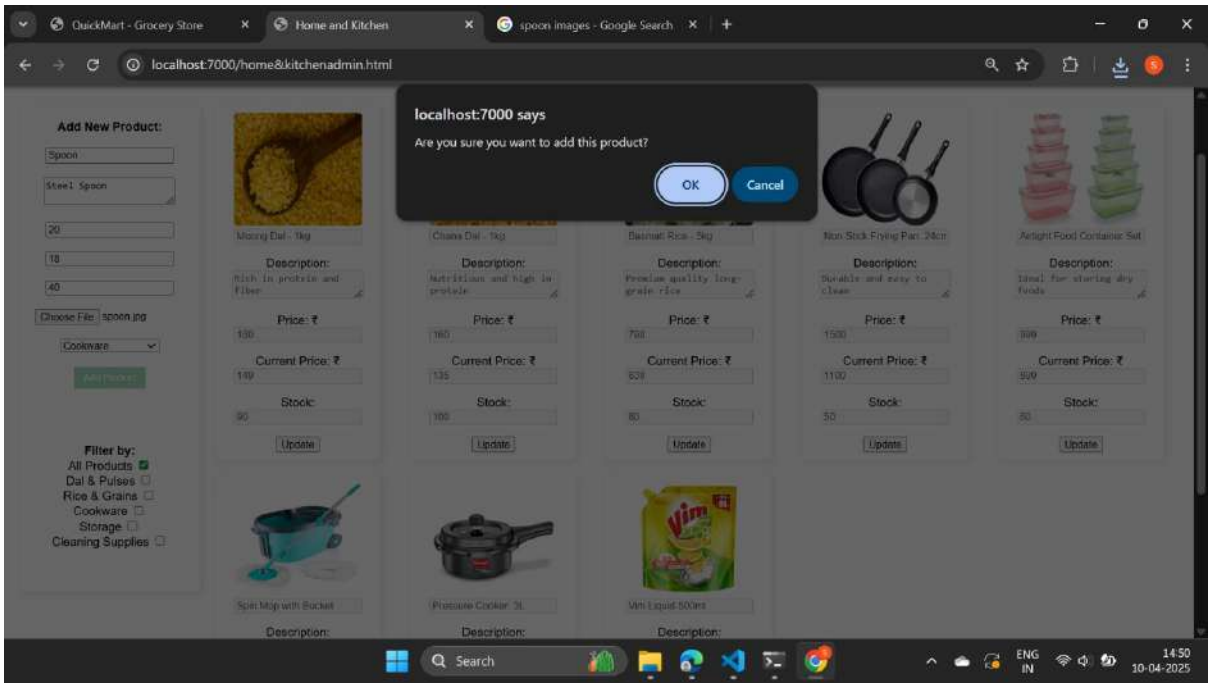


MY ORDERS :



ADMIN PAGE OUTPUT :




QuickMart - Grocery Store
Home and Kitchen
spoon images - Google Search
localhost:7000/home&kitchenadmin.html
Add New Product:
Spoon
Steel Spoon
20
18
40
Choose File
spoon.jpg
Cookware
Filter by:
All Products
Dal & Pulses
Rice & Grains
Cookware
Storage
Cleaning Supplies
localhost:7000 says
Are you sure you want to add this product?
OK
Cancel
Description:
Price: ₹
Current Price: ₹
Stock:
Update
Search
ENG
IN
14:50
10-04-2025